

5 Module 5: Threads

5.1 Benefits of Multithreading

5.1.1 Introduction

Multithreading is a programming and execution model that allows multiple threads (smaller units of a process) to run concurrently within a single process. This lecture explores the key benefits of multithreading in operating systems, analyzing each advantage in detail.

5.1.2 1. Responsiveness

5.1.2.1 Definition Responsiveness refers to the ability of an application to remain interactive and promptly respond to user inputs, even while performing computationally intensive or time-consuming tasks.

5.1.2.2 Key Points

- **Interactive Applications:** Users expect applications (e.g., word processors, web browsers) to handle multiple tasks simultaneously, such as:
 - Responding to keystrokes.
 - Displaying graphics.
 - Running background checks (e.g., spelling/grammar verification).
- **Single-Threaded Limitations:**
 - If a lengthy task (e.g., a complex computation triggered by a button click) executes in a single-threaded application, the **entire application freezes** until the task completes.
 - Users cannot interact with any part of the application during this block.
- **Multithreading Advantage:**
 - Long-running tasks execute in **separate threads**, allowing other threads to continue handling user interactions.
 - Example: A word processor can run a spell-check in the background while still accepting keystrokes.
 - Prevents **application latency** and improves user experience, particularly in **graphical user interfaces (GUIs)**.

5.1.2.3 Practical Implications

- Critical for **real-time systems** where delays are unacceptable (e.g., gaming, multimedia applications).
- Enables **non-blocking I/O operations**, where threads can wait for I/O (e.g., file reads) without stalling the entire application.

5.1.3 2. Resource Sharing

5.1.3.1 Definition Threads within the same process share resources (e.g., memory, files, variables) by default, eliminating the need for explicit **Inter-Process Communication (IPC)** mechanisms.

5.1.3.2 Key Points

- **Single-Threaded Processes:**
 - Require **IPC mechanisms** (e.g., shared memory, message queues, pipes) to communicate with other processes.
 - Programmers must explicitly implement these mechanisms using system calls, increasing complexity.
- **Multithreading Advantage:**
 - Threads in the same process **automatically share:**

- * **Code section** (executable instructions).
- * **Data section** (global/dynamically allocated variables).
- * **Open files** and other system resources (e.g., sockets, devices).
- **No IPC Overhead:** Threads communicate via shared variables/data structures without additional setup.
- Example: A multi-threaded server can share a global configuration table across all threads.

5.1.3.3 Practical Implications

- **Simplified Programming:** No need to manage IPC; threads inherently share the process's address space.
- **Efficient Data Exchange:** Shared memory access is faster than IPC methods like pipes or message queues.
- **Consistency:** All threads operate on the same data, reducing synchronization issues (though careful design is needed to avoid race conditions).

5.1.4 3. Economy

5.1.4.1 Definition Multithreading reduces computational and memory overhead compared to multi-processing, making it a cost-effective solution for concurrent task execution.

5.1.4.2 Key Points

- **Process Creation Overhead:**
 - Creating a new process involves:
 - * Allocating **separate memory** (address space).
 - * Duplicating resources (e.g., file descriptors, environment variables).
 - * High **system call overhead**.
 - **Expensive** in terms of time and resources.
- **Thread Creation Advantage:**
 - Threads share the **parent process's memory and resources**.
 - No need for separate address space allocation.
 - **Lower overhead** for creation and destruction.
- **Context Switching:**
 - **Process Context Switching:**
 - * Requires saving/restoring **entire process state** (e.g., memory mappings, registers).
 - * High latency due to **address space switches** (e.g., TLB flushes).
 - **Thread Context Switching:**
 - * Only saves/restores **thread-specific data** (e.g., registers, stack pointer).
 - * **Faster** and less resource-intensive.
- **Throughput:**
 - Assigning tasks to threads (instead of processes) **increases throughput** with lower resource consumption.

5.1.4.3 Practical Implications

- Ideal for **high-performance applications** (e.g., web servers, databases) where frequent task switching is required.
- Enables **fine-grained parallelism** without the cost of process management.

5.1.5 4. Scalability

5.1.5.1 Definition Multithreading leverages **multi-core/multi-processor architectures** to distribute workloads across available processing units, improving performance and efficiency.

5.1.5.2 Key Points

- **Single-Threaded Limitations:**
 - A single-threaded process can only utilize **one CPU core** at a time.
 - To parallelize work, multiple **processes** must be created, incurring high overhead (as discussed in “Economy”).
- **Multithreading Advantage:**
 - Threads of a process can run **concurrently on different cores**.
 - Example: A multi-threaded video encoder can assign frames to separate threads, each running on a distinct core.
 - **Computational Speedup:** Parallel execution reduces total completion time for CPU-bound tasks.
- **Modern Architectures:**
 - Multi-core CPUs and **SMP (Symmetric Multiprocessing)** systems are optimized for thread-level parallelism.
 - Threads avoid the **process creation bottleneck**, enabling efficient scaling.

5.1.5.3 Practical Implications

- **Load Balancing:** Threads can be dynamically assigned to cores based on workload (e.g., thread pools in servers).
- **Cost-Effective Parallelism:** Achieves performance gains without the overhead of multi-processing.
- **Future-Proofing:** Applications can scale with hardware advancements (e.g., increasing core counts).

5.1.6 Summary of Benefits

Benefit	Description	Key Advantage
Responsiveness	Enables concurrent task execution, preventing application freezes during long operations.	Improved user experience in interactive applications (e.g., GUIs).
Resource Sharing	Threads share memory and resources by default, eliminating IPC overhead.	Simplified programming and faster data exchange.
Economy	Lower overhead for thread creation/destruction and context switching compared to processes.	Higher throughput with reduced resource consumption.
Scalability	Threads can exploit multi-core architectures for parallel execution.	Computational speedup and efficient workload distribution.

5.1.7 Conclusion

Multithreading provides **responsiveness**, **resource efficiency**, **economic resource usage**, and **scalability**, making it indispensable for modern operating systems and applications. By leveraging threads, developers can: - Build **interactive, non-blocking applications**. - Share resources **seamlessly** without IPC. - Achieve **high performance** with lower overhead. - **Scale efficiently** across multi-core hardware.

Understanding these benefits is crucial for designing **efficient, concurrent systems** in operating system development.

5.2 Challenges of Multicore programming

5.2.1 Introduction

- Multicore programming involves designing applications that leverage multiple processing cores to execute tasks simultaneously.

- This lecture identifies and analyzes the key challenges associated with multicore programming.

5.2.2 1. Division of Tasks

5.2.2.1 Key Concept

- **Definition:** The process of identifying independent tasks within an application that can be executed in parallel across multiple cores.
- **Importance:** Only tasks that are **independent** of one another can be executed concurrently.

5.2.2.2 Challenges

- **Task Identification:**
 - Requires a thorough analysis of the application during the **design phase** to distinguish between independent and dependent tasks.
 - Failure to identify dependencies early leads to **execution conflicts** later, where tasks cannot run simultaneously due to hidden dependencies.
- **Consequences of Poor Division:**
 - If dependent tasks are incorrectly assumed to be independent, the program may fail or produce incorrect results when executed in parallel.

5.2.2.3 Best Practices

- Conduct a **detailed task dependency analysis** before implementation.
- Ensure that tasks are **logically separable** without requiring synchronization or communication during execution.

5.2.3 2. Balancing Task Workloads

5.2.3.1 Key Concept

- **Definition:** Ensuring that all parallel tasks contribute **equal computational effort** to avoid inefficient resource utilization.
- **Goal:** Prevent scenarios where one task is **computationally heavy** while another is **trivial**, leading to **underutilization of cores**.

5.2.3.2 Challenges

- **Uneven Work Distribution:**
 - A **trivial task** (e.g., a simple calculation) assigned to a separate thread wastes a core's processing power.
 - Example: If one thread performs a complex matrix multiplication while another merely increments a counter, the latter **blocks a core unnecessarily**.
- **Resource Blocking:**
 - A core assigned to a trivial task is **effectively idle**, reducing the overall efficiency of the multicore system.

5.2.3.3 Best Practices

- **Avoid trivial tasks:** Combine minor operations into larger tasks to ensure meaningful workload distribution.
- **Workload Analysis:** Quantify the computational load of each task to achieve **balanced parallelism**.

5.2.4 3. Splitting of Data

5.2.4.1 Key Concept

- **Definition:** Partitioning a dataset into segments that can be processed independently by different threads/cores.
- **Purpose:** Enable parallel execution by assigning **non-overlapping data segments** to separate tasks.

5.2.4.2 Challenges

- **Data Partitioning Complexity:**
 - Requires understanding the **data access patterns** of each task.
 - Example: In a large dataset (e.g., an array or database), each thread may need a **contiguous or logically grouped subset** of the data.
- **Incorrect Partitioning:**
 - If data is not split correctly, threads may **compete for the same data**, leading to **race conditions** or **inefficient memory access**.

5.2.4.3 Best Practices

- **Analyze Data Dependencies:** Ensure that partitioned data does not require cross-thread communication.
- **Use Efficient Data Structures:** Employ structures that support **locality of reference** (e.g., dividing an array into equal chunks).
- **Dynamic Partitioning:** For irregular workloads, use techniques like **work stealing** to balance data distribution at runtime.

5.2.5 4. Identifying Data Dependencies

5.2.5.1 Key Concept

- **Definition:** Detecting scenarios where tasks rely on **shared data** or **sequential execution**, making parallelism unsafe or impossible.
- **Types of Dependencies:**
 1. **Computational Dependency:** The output of one task is the input to another (e.g., Task T1's result is needed by Task T2).
 2. **Data Access Dependency:** Multiple tasks read/write the same data, requiring synchronization.

5.2.5.2 Challenges

- **Sequential Constraints:**
 - If Task T1 must complete before Task T2 can start (e.g., T2 uses T1's output), **parallel execution is invalid**.
 - Example: A pipeline where each stage depends on the previous stage's output.
- **Concurrent Data Modification:**
 - **Race Conditions:** Multiple threads writing to the same data simultaneously can **corrupt data**.
 - **Read-After-Write Dependencies:** If a read operation must occur **after** a write, unsynchronized access leads to **inconsistent results**.
- **Shared Data Conflicts:**
 - Example: Two threads updating a shared variable without locks may result in **lost updates** or **inconsistent states**.

5.2.5.3 Best Practices

- **Dependency Graphs:** Model tasks and data flows to identify dependencies visually.
- **Synchronization Mechanisms:** Use **locks, semaphores, or barriers** to enforce order where necessary.
- **Immutable Data:** Where possible, design tasks to work on **read-only copies** of data to avoid conflicts.
- **Static Analysis Tools:** Employ tools to detect potential data races during development.

5.2.6 5. Testing and Debugging

5.2.6.1 Key Concept

- **Definition:** Ensuring correctness and reliability in multicore programs by validating all possible **execution paths** and **thread interactions**.
- **Complexity:** Multithreaded applications have **exponentially more execution paths** than single-threaded programs.

5.2.6.2 Challenges

- **Non-Determinism:**
 - Thread scheduling is **non-deterministic**; the same input may produce different outputs across runs due to **race conditions** or **timing variations**.
- **Exhaustive Testing:**
 - Single-threaded programs have finite paths (e.g., branches in **if-else** or loops).
 - Multithreaded programs introduce **interleaved executions**, making it **impossible to test all paths exhaustively**.
- **Reproducibility:**
 - Bugs (e.g., deadlocks, data races) may be **hard to reproduce** due to timing dependencies.
- **Debugging Complexity:**
 - Traditional debugging tools (e.g., breakpoints) may **disrupt timing**, masking or altering concurrency issues.

5.2.6.3 Best Practices

- **Stress Testing:** Run the program under **high concurrency** to expose race conditions.
- **Static Analysis:** Use tools like **ThreadSanitizer** or **Helgrind** to detect data races.
- **Formal Verification:** Apply model checking to verify correctness for critical sections.
- **Logging and Tracing:** Record thread interactions to diagnose **deadlocks** or **starvation**.
- **Deterministic Testing:** Use controlled scheduling (e.g., **record-and-replay debugging**) to reproduce concurrency bugs.

5.2.7 Summary of Challenges

Challenge	Description	Key Risk
Division of Tasks	Identifying independent tasks for parallel execution.	Hidden dependencies cause execution failures.
Balancing Workloads	Ensuring equal computational load across tasks.	Core underutilization or bottlenecks.
Splitting Data	Partitioning data for parallel access without overlap.	Race conditions or inefficient memory use.
Data Dependencies	Detecting shared data or sequential constraints between tasks.	Data corruption or incorrect results.

Challenge	Description	Key Risk
Testing and Debugging	Validating correctness across all possible thread interleavings.	Unreproducible bugs or missed edge cases.

5.2.8 Conclusion

- Multicore programming offers **performance benefits** but introduces **significant complexity** in design, implementation, and verification.
- Addressing these challenges requires:
 - **Careful task and data analysis** during design.
 - **Balanced workload distribution** to maximize core utilization.
 - **Robust synchronization** to handle dependencies.
 - **Rigorous testing** to ensure correctness across all execution paths.
- Mastery of these challenges is essential for developing **efficient, reliable multicore applications**.

5.3 Example of multithreaded program

5.3.1 Introduction

This lecture provides a practical example of a **multithreaded program** using the **POSIX Threads (pthreads)** library. The program demonstrates: - Thread creation (`pthread_create`). - Thread attributes initialization (`pthread_attr_init`). - Thread synchronization (`pthread_join`). - Thread termination (`pthread_exit`). - Shared and local variables in multithreading. - Compilation and execution of a pthreads program.

5.3.2 Program Overview

The example program illustrates how multiple threads can be created, execute concurrently, and synchronize with the main thread. Below is a detailed breakdown of the program structure and logic.

5.3.3 Header Files and Dependencies

The program includes three essential header files: 1. `pthread.h` - Provides declarations for all pthreads functions (e.g., `pthread_create`, `pthread_join`, `pthread_exit`). - Required for any multithreaded program using POSIX Threads. 2. `stdio.h` - Provides input/output functions (e.g., `printf`). 3. `stdlib.h` - Provides general utility functions (e.g., memory allocation, process control).

Note: The `-pthread` compiler flag **must** be used when compiling pthreads programs to link the pthreads library correctly.

5.3.4 Global Variable

- A **global variable** `x` is declared and initialized to `10`.

```
int x = 10;
```

- **Purpose:**
 - Shared among **all threads** in the program.
 - Demonstrates how threads can access and modify shared data.
 - In this example, `x` is read by both child threads but not modified.

5.3.5 Main Function

The main function orchestrates the creation, execution, and synchronization of threads.

5.3.5.1 1. Variable Declarations

- **Thread Identifiers** (`pthread_t`):
 - `tid1` and `tid2` store the **unique identifiers** of the two child threads.
 - Used to reference threads in functions like `pthread_join`.
- **Thread Attributes** (`pthread_attr_t`):
 - `attr1` and `attr2` are **attribute objects** that define thread properties (e.g., stack size, scheduling policy).
 - Initialized with default values using `pthread_attr_init`.
- **Local Variables:**
 - `a = 10` and `b = 20` are passed as arguments to the child threads.

5.3.5.2 2. Initializing Thread Attributes

- `pthread_attr_init` initializes thread attribute objects with default values.

```
pthread_attr_init(&attr1);  
pthread_attr_init(&attr2);
```

- **Purpose:**
 - Ensures threads are created with standard attributes unless explicitly modified.
 - If custom attributes (e.g., detached state, stack size) are needed, they can be set after initialization.

5.3.5.3 3. Thread Creation (`pthread_create`) Two child threads are created using `pthread_create`:

```
pthread_create(&tid1, &attr1, threadrun, &a);  
pthread_create(&tid2, &attr2, threadrun, &b);
```

5.3.5.3.1 Parameters of `pthread_create`:

1. `&tid1 / &tid2`
 - Pointer to a `pthread_t` variable where the **thread ID** is stored.
2. `&attr1 / &attr2`
 - Pointer to the **thread attributes** object (default in this case).
3. `threadrun`
 - The **function** that the thread will execute upon creation.
4. `&a / &b`
 - **Argument** passed to `threadrun` (address of `a` or `b`).

5.3.5.3.2 Key Observations:

- Both threads execute the **same function** (`threadrun`) but with **different arguments** (`&a` and `&b`).
- The **main thread** continues execution after creating the child threads unless synchronized (e.g., with `pthread_join`).

5.3.5.4 4. Thread Synchronization (`pthread_join`) The main thread waits for both child threads to complete using `pthread_join`:


```
pthread_join(tid1, NULL);  
pthread_join(tid2, NULL);
```

5.3.5.4.1 Parameters of pthread_join:

1. tid1 / tid2
 - The **thread ID** of the child thread to wait for.
2. NULL
 - Indicates the main thread **ignores the exit status** of the child thread.
 - If a pointer is provided, it stores the thread's return value (e.g., from `pthread_exit`).

5.3.5.4.2 Purpose:

- Ensures the main thread **does not exit prematurely** before child threads complete.
- Guarantees that “**Main Thread Exiting**” is printed **last** in the output.

5.3.6 Thread Function (threadrun)

The `threadrun` function is executed by both child threads. It performs the following steps:

5.3.6.1 1. Local Variable Declaration

- `sum` is a **local variable** unique to each thread.
 - Not shared between threads (unlike the global variable `x`).

5.3.6.2 2. Argument Handling

- The function receives a `void*` argument (generic pointer).
- The argument is **typecast** to an `int*` to access the passed value:

```
int *val = (int *)arg;
```

- **Purpose:**
 - Allows the thread to use the value (`a` or `b`) passed during creation.

5.3.6.3 3. Computation

- The thread computes the sum of:
 - The **global variable** `x` (shared, value = 10).
 - The **passed argument** (`*val`, either 10 or 20).

- Result is stored in the local variable `sum`:

```
sum = x + *val;
```

- **Output:**
 - **First thread (tid1):** `sum = 10 (x) + 10 (a) = 20`
 - **Second thread (tid2):** `sum = 10 (x) + 20 (b) = 30`

5.3.6.4 4. Printing and Termination

- The thread prints:
 - The computed `sum`.
 - The message “**Thread Exiting**”.
- The thread terminates using `pthread_exit`:

```
pthread_exit(0);
```

- 0 indicates **normal termination** (no errors).

5.3.7 Program Execution Flow

1. Main Thread:

- Initializes thread attributes.
- Creates two child threads (`tid1` and `tid2`).
- Waits for both threads to finish (`pthread_join`).
- Prints “**Main Thread Exiting**” and exits.

2. Child Threads (`tid1` and `tid2`):

- Execute `threadrun` concurrently.
- Compute `sum` using shared (`x`) and local (`a/b`) data.
- Print results and terminate.

5.3.7.1 Key Concurrency Behavior:

- The **order of execution** of the child threads is **non-deterministic**.
 - Output may vary between runs (e.g., `tid1` could print before or after `tid2`).
- The **main thread always waits** for both child threads to finish before exiting.

5.3.8 Compilation and Execution

```
gcc threadprog.c -pthread
```

5.3.8.1 1. Compilation Command

- `-pthread` is **mandatory** for linking the pthreads library.
- Without it, the program may **fail to compile or behave unpredictably**.

```
./a.out
```

5.3.8.2 2. Execution Command

5.3.8.3 3. Sample Output

```
Sum = 20
Thread Exiting
Sum = 30
Thread Exiting
Main Thread Exiting
```

Alternative Possible Output (due to concurrency):

Sum = 30
Thread Exiting
Sum = 20
Thread Exiting
Main Thread Exiting

5.3.8.4 Guarantees:

- “Main Thread Exiting” is always last (due to `pthread_join`).
- The order of the child threads’ output is **not guaranteed**.

5.3.9 Key Concepts Demonstrated

1. **Thread Creation (`pthread_create`):**
 - Spawns new threads with specified attributes and functions.
2. **Thread Attributes (`pthread_attr_t`):**
 - Default attributes are used unless customized.
3. **Shared vs. Local Variables:**
 - **Global variables** (`x`) are shared across threads.
 - **Local variables** (`sum`, `val`) are unique to each thread.
4. **Thread Synchronization (`pthread_join`):**
 - Ensures the main thread waits for child threads to complete.
5. **Thread Termination (`pthread_exit`):**
 - Child threads exit gracefully with a status code.
6. **Non-Deterministic Execution:**
 - Thread scheduling is handled by the OS; output order may vary.

5.3.10 Summary of Pthreads Functions Used

Function	Purpose
<code>pthread_create</code>	Creates a new thread with specified attributes and start function.
<code>pthread_attr_init</code>	Initializes thread attributes with default values.
<code>pthread_join</code>	Blocks the calling thread until the specified thread terminates.
<code>pthread_exit</code>	Terminates the calling thread and returns a status value.

5.3.11 Important Notes

- **Compilation Requirement:**
 - Always use `-pthread` when compiling pthreads programs.
- **Thread Safety:**
 - Shared variables (e.g., `x`) must be accessed carefully to avoid race conditions (not an issue in this example since `x` is only read).
- **Concurrency Behavior:**
 - The order of thread execution is **not guaranteed** unless explicitly synchronized.

5.4 Many-to-Many Model

5.4.1 1. Introduction to the Many-to-Many Multithreading Model

- The **many-to-many (M:N) multithreading model** is a threading architecture where **multiple user-level threads (ULTs)** are mapped to **multiple kernel-level threads (KLT)**.

- The number of **user-level threads** is typically **equal to or greater than** the number of **kernel-level threads** allocated to an application.
- The model derives its name from the **many-to-many relationship** between user threads and kernel threads.

5.4.2 2. Key Characteristics of the Many-to-Many Model

5.4.2.1 2.1 Thread Mapping and Allocation

- The **operating system (OS)** allocates a **fixed number of kernel-level threads per application**.
 - This number is **predefined** and may be:
 - * **OS-specific** (determined by the operating system's design).
 - * **System architecture-specific** (dependent on hardware constraints).
 - * **Application-specific** (varies between different applications on the same machine).
- Example:
 - An application may have **4 user-level threads** mapped to **3 kernel-level threads** (as shown in the lecture's diagram).
 - The OS does **not** necessarily allocate an equal number of kernel threads to user threads.

5.4.2.2 2.2 Thread Execution Spaces

- **User-level threads (ULTs)** execute in **user space**.
- **Kernel-level threads (KLTs)** execute in **kernel space**.

5.4.2.3 2.3 Multiplexing of Threads

- Multiple user-level threads **multiplex** (share) a smaller pool of kernel-level threads.
- This allows **concurrency** even when the number of kernel threads is limited.

5.4.3 3. Operational Details of the Many-to-Many Model

5.4.3.1 3.1 Handling Blocking System Calls

- If a **user-level thread** performs a **blocking system call** (e.g., I/O operation), the **corresponding kernel-level thread also blocks**.
- However, since **other kernel-level threads** remain available, the OS can **schedule another kernel thread** to continue execution.
 - This prevents the **entire application from stalling** due to a single blocked thread.

5.4.3.2 3.2 Unrestricted User-Level Thread Creation

- **Application programmers** can create an **unlimited number of user-level threads** without direct constraints from the OS.
 - Unlike the **one-to-one model**, where the number of user threads is limited by the number of kernel threads, the **many-to-many model removes this restriction**.

5.4.3.3 3.3 Concurrency and Parallelism

- The model **supports concurrency** by allowing multiple threads to execute concurrently on available kernel threads.
- If the system has **multiple processing cores**, the model can also enable **parallelism** (simultaneous execution of threads on different cores).
- **Advantages over other models:**
 - **Better than the many-to-one model** (which lacks true parallelism due to a single kernel thread).

- **More flexible than the one-to-one model** (which restricts the number of user threads).

5.4.3.4 3.4 Overhead and Resource Management

- **Kernel-level threads incur creation and management overhead** (e.g., context switching, scheduling).
 - To optimize performance, the OS **limits the number of kernel threads** per application.
 - The exact number may depend on:
 - * **Machine specifications** (e.g., CPU cores, memory).
 - * **Application requirements** (e.g., some applications may receive more kernel threads than others).

5.4.4 4. Comparison with Other Multithreading Models

Feature	Many-to-Many Model	One-to-One Model	Many-to-One Model
User threads per KLT	Many	One	Many
Kernel threads per ULT	Shared (multiplexed)	One per ULT	One (shared)
Concurrency support	Yes	Yes	Limited
Parallelism support	Yes (if cores available)	Yes	No
Blocking impact	Only blocks one KLT	Blocks entire ULT	Blocks all ULTs
Thread creation limit	No limit on ULTs	Limited by KLTs	No limit on ULTs
Overhead	Moderate (fixed KLTs)	High (per ULT)	Low (single KLT)

5.4.4.1 4.1 Key Differences from the One-to-One Model

- **One-to-one model:**
 - Each user thread is **directly mapped to a unique kernel thread**.
 - **Limits the number of user threads** (since each requires a kernel thread).
 - **Higher overhead** due to per-thread kernel management.
- **Many-to-many model:**
 - **No direct 1:1 mapping**; multiple user threads share kernel threads.
 - **No hard limit on user threads**.
 - **Better resource utilization** (fewer kernel threads needed).

5.4.4.2 4.2 Key Differences from the Many-to-One Model

- **Many-to-one model:**
 - **All user threads map to a single kernel thread**.
 - **No true parallelism** (only one thread can execute at a time).
 - **Blocking system calls stall the entire process**.
- **Many-to-many model:**
 - **Multiple kernel threads allow parallel execution**.
 - **Blocking a user thread does not halt the entire application**.

5.4.5 5. The Two-Level Multithreading Model (Extension of Many-to-Many)

5.4.5.1 5.1 Definition and Overview

- The **two-level model** is an **extension of the many-to-many model** with an **additional feature**:
 - It allows **some user-level threads to be directly mapped to individual kernel-level threads** (similar to the one-to-one model).
 - The remaining user threads **multiplex over a pool of kernel threads** (many-to-many).

5.4.5.2 5.2 Thread Mapping in the Two-Level Model

- **Example (from lecture diagram):**
 - **4 user-level threads** multiplex over **3 kernel-level threads** (many-to-many).
 - **1 additional user-level thread** is **directly mapped to a dedicated kernel thread** (one-to-one).
- This creates a **hybrid model** combining:
 - **Many-to-many multiplexing** (for most threads).
 - **One-to-one mapping** (for select threads requiring dedicated kernel resources).

5.4.5.3 5.3 Purpose of One-to-One Mapping in Two-Level Model

- **Why map a user thread directly to a kernel thread?**
 - To **ensure uninterrupted execution** for critical threads.
 - To **avoid multiplexing overhead** for performance-sensitive operations.
 - To **leverage parallelism** more effectively for specific threads.

5.4.5.4 5.4 Execution Spaces in Two-Level Model

- **User-level threads** still run in **user space**.
- **Kernel-level threads** run in **kernel space**.
- The model **retains the benefits of both many-to-many and one-to-one models**.

5.4.6 6. Summary of Key Concepts

- The **many-to-many model** allows **multiple user threads to share multiple kernel threads**, enabling **concurrency and parallelism** without restricting user thread creation.
- The **operating system preallocates a fixed number of kernel threads per application**, which may vary based on system or application needs.
- **Blocking system calls** only affect the associated kernel thread, allowing other threads to continue execution.
- The **two-level model** extends the many-to-many approach by **allowing some user threads to have a direct one-to-one mapping with kernel threads**, combining flexibility with performance optimization.
- **Advantages over other models:**
 - **More scalable** than one-to-one (no hard limit on user threads).
 - **More efficient** than many-to-one (supports parallelism).
 - **More flexible** than pure many-to-many (allows dedicated kernel threads when needed).

5.5 Many-to-One Model

5.5.1 Introduction to Multi-Threading Models

- **Purpose of Multi-Threading Models:**
 - The **kernel** does **not** recognize **user-level threads (ULTs)**.
 - The kernel can **only recognize and schedule kernel-level threads (KLT)**.
 - **Problem:** How can user-level threads (created in application programs) be scheduled by the kernel?
 - **Solution:** A **mapping mechanism** is required to associate user-level threads with kernel-level threads.
 - **Multi-threading models** define how user-level threads are mapped to kernel-level threads.

5.5.2 Types of Multi-Threading Models

Three primary models exist: 1. **Many-to-One Model** 2. **One-to-One Model** 3. **Many-to-Many Model**

5.5.3 Many-to-One Model

5.5.3.1 Definition

- **Mapping:** Multiple user-level threads (ULTs) are mapped to a single kernel-level thread (KLT).
- **Management:**
 - User-level threads are managed by the **thread library in user space**.
 - The **kernel-level thread** is managed by the **kernel**.
 - **Key Limitation:** Only **one kernel-level thread** exists for all user-level threads in the process.

5.5.3.2 Visual Representation

- An **application** contains **multiple user-level threads** (e.g., four threads).
- **All user-level threads** are mapped to **one kernel-level thread**.
- **User-level threads** execute in **user space**.
- The **kernel-level thread** executes in **kernel space**.

5.5.3.3 Characteristics

5.5.3.3.1 1. Unrestricted User-Level Thread Creation

- **No limit** on the number of user-level threads an application programmer can create.
- Programmers can spawn as many threads as needed for the application's logic.

5.5.3.3.2 2. Blocking Behavior

- **Critical Limitation:** If **any single user-level thread blocks** (e.g., due to a **blocking system call** or I/O operation):
 - The **entire process blocks**.
 - **Reason:** The blocking user-level thread causes the **associated kernel-level thread to block**.
 - Since there is **only one kernel-level thread**, the **entire application halts**.

5.5.3.3.3 3. Lack of Parallelism and Concurrency

- **No true parallelism:**
 - Even if multiple user-level threads exist, they **cannot run in parallel** on a **multicore system**.
 - Only **one thread** (the kernel-level thread) is scheduled by the kernel at any time.
- **No concurrency:**
 - The model does **not** support overlapping execution of threads.
 - **All threads execute sequentially** under the single kernel-level thread.

5.5.3.3.4 4. Inefficient Utilization of Multicore Systems

- **Cannot leverage multiple CPU cores** because only one kernel-level thread exists.
- **Performance bottleneck:** Threads cannot distribute workload across available cores.

5.5.3.4 Advantages (Limited)

- **Simplicity:**
 - Easier to implement due to minimal kernel involvement.
 - Thread management is handled entirely in **user space** (no kernel intervention required for thread scheduling).
- **Low overhead:**

- No context-switching overhead between kernel-level threads (since there is only one).

5.5.3.5 Disadvantages (Significant)

1. **Poor Performance:**
 - **Single point of failure:** One blocked thread halts the entire process.
 - **No parallel execution** even on multicore systems.
2. **Lack of Scalability:**
 - Cannot efficiently utilize modern **multiprocessor/multicore** architectures.
3. **Obsolete in Modern Systems:**
 - **Rarely used** in contemporary operating systems due to severe limitations.
 - Replaced by more efficient models (e.g., **many-to-many** or **one-to-one**).

5.5.3.6 Use Cases (Historical Context)

- Early threading implementations where **kernel support for threads was minimal**.
- Systems with **limited hardware resources** where simplicity was prioritized over performance.

5.5.4 Summary of Key Points

- The **many-to-one model** maps **multiple user-level threads to a single kernel-level thread**.
- **Thread management** is handled by the **user-space thread library**, while the kernel manages the single kernel thread.
- **Major drawback: Blocking one thread blocks the entire process.**
- **No parallelism or concurrency** due to the single kernel thread.
- **Inefficient for multicore systems**-cannot distribute workload.
- **Rarely used today** due to superior alternatives (one-to-one, many-to-many).

5.5.5 Conclusion

- The **many-to-one model** is a **basic but restrictive** threading model.
- While it offers **simplicity and low overhead**, its **inability to support parallelism and concurrency** makes it **impractical for modern systems**.
- Understanding this model helps contrast it with more advanced models (one-to-one, many-to-many) covered in subsequent lectures.

5.6 Motivation of Multithreading

5.6.1 1. Introduction to Multithreading

5.6.1.1 1.1 Overview of Modern Software Applications

- Modern software applications are **predominantly multi-threaded**.
- **Single-threaded applications** are rare in contemporary computing.
- Multi-threaded applications consist of **multiple threads of execution**, each performing distinct tasks.

5.6.1.2 1.2 Key Benefits of Multithreading

- **Concurrent User Interaction:**
 - Users can interact with **multiple aspects of the same application simultaneously** due to parallel execution of threads.
 - Example: A web browser can download files, render a webpage, and play audio concurrently.
- **Parallel Execution on Multi-Core Systems:**

- Threads can execute **in parallel** if the system has **multiple processors or cores**, improving performance.
- **Efficient Task Handling for Repetitive Operations:**
 - If an application must perform **similar tasks repeatedly**, multithreading allows each task to run in a **separate thread** rather than launching multiple instances of the application.
 - Example: A web server handling multiple client requests simultaneously.

5.6.2 2. Why Multithreading is Necessary

5.6.2.1 2.1 Limitations of Single-Threaded Applications

- A **single-threaded process** can only execute **one task at a time**.
- If a task is **blocked** (e.g., waiting for I/O, network response), the entire application **stalls**.
- **Poor responsiveness** in interactive applications (e.g., GUIs freezing during file operations).

5.6.2.2 2.2 Advantages Over Single-Process Solutions

- **Resource Efficiency:**
 - Threads **share the same memory space**, reducing memory overhead compared to multiple processes.
 - **Context switching** between threads is **faster** than between processes.
- **Performance Gains:**
 - **Parallelism** on multi-core systems leads to **faster execution** of independent tasks.
 - **Reduced latency** in applications requiring concurrent operations (e.g., real-time systems).

5.6.3 3. Case Study: Client-Server Architecture

5.6.3.1 3.1 Problem with Single-Process Servers

- **Scenario:** Hundreds/thousands of clients send **simultaneous requests** to a server.
- **Single-Process Server Behavior:**
 - The server processes **one request at a time** (sequentially).
 - Clients must **wait in a queue**, leading to **high latency** and **poor scalability**.
 - **Bottleneck:** The server cannot handle multiple requests concurrently.

5.6.3.2 3.2 Multiprocessing Approach (Process-per-Request)

- **Mechanism:**
 1. A **client** sends a request to the **server process**.
 2. The server **spawns a new child process** (identical to the parent) to handle the request.
 3. The **child process** services the request and sends a response back to the client.
 4. The **original server process** continues listening for new requests.
- **Advantages:**
 - **Concurrent request handling:** Multiple clients can be serviced simultaneously.
- **Disadvantages:**
 - **High overhead:** **Process creation is expensive** (memory allocation, OS resources).
 - **Scalability issues:** Spawning thousands of processes is **resource-intensive** and **inefficient**.

5.6.3.3 3.3 Multithreading Approach (Thread-per-Request)

- **Mechanism:**
 1. A **client** sends a request to the **server process**.
 2. The **main thread** of the server creates a **new thread** to handle the request.

3. The **new thread** processes the request and sends a response to the client.
 4. The **main thread** remains free to accept new requests.
- **Advantages Over Multiprocessing:**
 - **Lower overhead:** Thread creation is significantly cheaper than process creation.
 - **Shared memory:** Threads within the same process **share data**, reducing memory usage.
 - **Faster context switching:** Threads switch more quickly than processes.
 - **Scalability:**
 - Can handle **thousands of concurrent requests** with **minimal resource consumption** compared to multiprocessing.

5.6.4 4. Comparison: Multiprocessing vs. Multithreading in Client-Server Models

Aspect	Multiprocessing (Process-per-Request)	Multithreading (Thread-per-Request)
Resource Overhead	High (each process has its own memory)	Low (threads share memory)
Creation Cost	Expensive (OS-level process creation)	Cheap (thread creation is lightweight)
Context Switching	Slow (process switches are costly)	Fast (thread switches are efficient)
Scalability	Poor (limited by system process limits)	High (can handle many threads)
Data Sharing	Requires IPC (Inter-Process Communication)	Direct memory access (shared space)
Fault Isolation	High (one process crash doesn't affect others)	Low (one thread crash can affect the process)

5.6.5 5. Real-World Applications of Multithreading

5.6.5.1 5.1 Web Servers (e.g., Apache, Nginx)

- Handle **thousands of HTTP requests concurrently** using threads.
- Example: A single-threaded server would fail under heavy load; multithreading ensures **responsiveness**.

5.6.5.2 5.2 Graphical User Interfaces (GUIs)

- **Main thread** handles user input (e.g., button clicks).
- **Background threads** perform long-running tasks (e.g., file downloads, image processing).
- Prevents **UI freezing** during intensive operations.

5.6.5.3 5.3 Database Systems

- **Multiple queries** can execute simultaneously via threads.
- Example: A database server handling **concurrent read/write operations** from different clients.

5.6.5.4 5.4 Real-Time Systems (e.g., Gaming, Multimedia)

- **Separate threads** for:
 - Rendering graphics.
 - Processing audio.
 - Handling user input.
- Ensures **smooth performance** without lag.

5.6.5.5 5.5 Parallel Computing (e.g., Scientific Simulations)

- **Divide-and-conquer** approach: Split tasks across threads for **faster execution** on multi-core CPUs.

5.6.6 6. Summary of Key Takeaways

1. **Multithreading enables concurrent execution** of tasks within a single process.
2. **Modern applications are multi-threaded** due to **performance, responsiveness, and efficiency** requirements.
3. **Client-server architectures benefit significantly** from multithreading:
 - **Multiprocessing is resource-heavy and scalability-limited.**
 - **Multithreading provides low-overhead concurrency with shared memory.**
4. **Thread creation is cheaper than process creation**, making multithreading ideal for high-load scenarios.
5. **Real-world use cases** include web servers, GUIs, databases, and real-time systems.

5.6.7 7. Conclusion

Multithreading is a **fundamental concept** in modern operating systems and application design. It addresses the **limitations of single-threaded processes** by enabling **concurrent task execution** with **minimal overhead**. The **client-server example** demonstrates how multithreading **improves scalability and efficiency** compared to traditional multiprocessing approaches. Understanding multithreading is **essential** for designing **high-performance, responsive, and scalable** software systems.

5.7 One-to-One Model

5.7.1 Introduction to the One-to-One Model

- The **one-to-one model** is a **multithreading model** in which **each user-level thread (ULT)** is mapped to a **separate kernel-level thread (KLT)**.
- This model ensures a **direct one-to-one correspondence** between user threads and kernel threads.
- If a multithreaded application contains **N user-level threads**, the operating system (OS) allocates **N corresponding kernel-level threads**.

5.7.2 Thread Mapping in the One-to-One Model

5.7.2.1 Basic Structure

- A **multithreaded application** executes in **user space**, containing multiple **user-level threads (ULTs)**.
- Each **ULT** is **directly associated** with a **unique kernel-level thread (KLT)** executing in **kernel space**.
- Example:
 - If an application has **4 user-level threads**, the OS allocates **4 kernel-level threads**, each mapped to one ULT.

5.7.2.2 Visual Representation

- **User Space:**
 - Contains the **multithreaded application** with its **user-level threads (ULTs)**.
- **Kernel Space:**
 - Contains the **kernel-level threads (KLTs)**, each tied to a specific ULT.
- The **one-to-one relationship** ensures that **each ULT has its own dedicated KLT**.

5.7.3 Behavior of Threads in the One-to-One Model

5.7.3.1 Blocking and Concurrency

- If **one user-level thread blocks** (e.g., due to I/O or synchronization), **only its corresponding kernel-level thread blocks**.

- **Other threads in the application remain unaffected** and can continue execution.
 - Example:
 - * If **Thread 1 (ULT)** blocks while waiting for I/O, **Thread 2 (ULT)** can still run on its assigned **KLT**.
- This **prevents the entire application from blocking**, unlike in the **many-to-one model**, where a single blocked ULT halts the entire process.

5.7.3.2 Advantages Over Many-to-One Model

1. **Higher Concurrency:**
 - Multiple threads can execute **simultaneously** if the system supports **multiprocessing or multicore architectures**.
2. **Better Parallelism:**
 - On **multicore/multiprocessor systems**, different threads can run on **different CPU cores** in parallel.
3. **Efficient Utilization of Modern Architectures:**
 - The model **maximizes CPU usage** by allowing true parallel execution of threads.

5.7.4 Performance and Overhead Considerations

5.7.4.1 Thread Creation Overhead

- **Every new user-level thread (ULT) requires the creation of a new kernel-level thread (KLT).**
- **Kernel-level thread creation involves significant overhead** because:
 - The OS must allocate **kernel resources** (e.g., stack, thread control block).
 - System calls and context switches are **more expensive** than pure user-space operations.

5.7.4.2 Mitigating Overhead

- To **reduce excessive overhead**, some systems **limit the number of threads per process**.
 - Example:
 - * An application may be **restricted to creating only 100 threads**, preventing excessive KLT creation.
- This **balances performance and resource usage** while maintaining the benefits of the one-to-one model.

5.7.5 Comparison with Other Multithreading Models

Feature	One-to-One Model	Many-to-One Model
Thread Mapping	1 ULT -> 1 KLT (direct mapping)	Multiple ULTs -> 1 KLT (shared mapping)
Blocking Behavior	Only the blocked thread stalls; others continue	Entire process blocks if one ULT blocks
Concurrency	High (true parallelism on multicore)	Limited (no parallelism, only concurrency)
Overhead	High (KLT creation per ULT)	Low (only one KLT per process)
Scalability	Excellent (scales with CPU cores)	Poor (no parallel execution)
Kernel Support	Requires kernel thread support	Can work without full kernel thread support

5.7.6 Use Cases and Applicability

- **Best suited for:**
 - **Multicore/multiprocessor systems** where **true parallelism** is desired.

- Applications requiring **high concurrency** (e.g., web servers, databases).
- Scenarios where **blocking one thread should not halt the entire process**.
- **Less ideal for:**
 - Systems with **limited kernel resources** (due to overhead).
 - Applications requiring **thousands of threads** (may hit thread limits).

5.7.7 Key Takeaways

1. **One-to-One Mapping:**
 - Each **user-level thread (ULT)** is bound to a **unique kernel-level thread (KLT)**.
2. **Independent Execution:**
 - Blocking one thread **does not affect others**, improving concurrency.
3. **Parallelism on Multicore:**
 - Enables **true parallel execution** on multiprocessor/multicore systems.
4. **Overhead Trade-off:**
 - **Higher performance** but **increased overhead** due to KLT creation.
5. **Thread Limits:**
 - Some systems **restrict the number of threads per process** to manage overhead.

5.7.8 Conclusion

- The **one-to-one model** provides **strong concurrency and parallelism** by mapping each user thread to a kernel thread.
- It is **highly effective on modern multicore systems** but introduces **overhead due to kernel thread management**.
- **Thread limits** may be imposed to **balance performance and resource usage**.

5.8 Parallelism vs Concurrency

5.8.1 1. Introduction to Key Concepts

5.8.1.1 1.1 Definitions

- **Parallelism:**
 - Refers to **performing multiple tasks simultaneously** (i.e., at the **same time**).
 - Requires **physical hardware support** (e.g., multi-processor or multi-core systems).
 - Each task runs on a **separate processing unit (CPU/core) concurrently in time**.
- **Concurrency:**
 - Refers to **supporting and allowing multiple tasks to make progress** within the **same span of time**.
 - Tasks **appear** to execute simultaneously but may not be **truly simultaneous**.
 - Achieved via **time-sharing** (rapid switching between tasks).
 - Does **not** require multiple CPUs/cores; can be implemented on **single-core systems**.

5.8.1.2 1.2 Key Distinction

Aspect	Parallelism	Concurrency
Execution	Tasks run simultaneously (true overlap).	Tasks appear simultaneous (illusion via switching).
Hardware	Requires multi-core/multi-processor .	Works on single-core systems.
Performance	True speedup (tasks complete faster).	Perceived speedup (tasks progress together).

Aspect	Parallelism	Concurrency
Implication	Parallelism implies concurrency.	Concurrency does not imply parallelism.

5.8.2 2. Hardware Requirements

5.8.2.1 2.1 Parallelism

- **Mandatory:** Multi-core or multi-processor system.
 - Example: Dual-core CPU can run **two tasks truly in parallel** (one per core).
- **Limitation:** Cannot exist on a **single-core system**.

5.8.2.2 2.2 Concurrency

- **Works on:**
 - **Single-core systems** (via time-slicing).
 - **Multi-core systems** (each core can handle concurrent tasks).
- **Mechanism:** Rapid **context switching** between tasks.

5.8.3 3. Relationship Between Parallelism and Concurrency

5.8.3.1 3.1 Implication Rules

1. **Parallelism $\Rightarrow$ Concurrency:**
 - If a system supports **parallelism**, it **must** support concurrency.
 - Example: A dual-core system running two tasks in parallel is **also concurrent** (each core may switch between multiple tasks).
2. **Concurrency $\Rightarrow$ not Parallelism:**
 - A system can be **concurrent without being parallel**.
 - Example: A single-core system switching between tasks is concurrent but **not parallel**.

5.8.3.2 3.2 Practical Example

- **Parallel + Concurrent System** (Multi-core):
 - **Parallelism:** Two tasks run on two cores **simultaneously**.
 - **Concurrency:** Each core may switch between **multiple tasks** (e.g., Core 0 runs T1 and T3 alternately; Core 1 runs T2 and T4 alternately).
- **Concurrent-only System** (Single-core):
 - **No parallelism:** Only one task runs at any instant.
 - **Concurrency:** Tasks **appear** to run together due to rapid switching.

5.8.4 4. Execution Models

5.8.4.1 4.1 Concurrency on a Single-Core System

5.8.4.1.1 Mechanism: Time-Slicing

1. The CPU executes **Task 1 (T1)** for a **short time slice**.
2. Switches to **Task 2 (T2)**, executes for a time slice.
3. Continues to **Task 3 (T3)**, **Task 4 (T4)**, then **cycles back to T1**.
4. **Illusion of parallelism:** Switching is so fast that users perceive all tasks as running “at the same time.”

5.8.4.1.2 Key Observations

- **No true simultaneity:** Only **one task** runs at any given moment.
- **Progress:** All tasks **advance** within the same time span (e.g., 1 second), but not simultaneously.
- **Example:**
 - Time 0-0.1s: T1 runs.
 - Time 0.1-0.2s: T2 runs.
 - Time 0.2-0.3s: T3 runs.
 - Time 0.3-0.4s: T4 runs.
 - Time 0.4-0.5s: Back to T1.
 - **User perception:** All tasks are “running together.”

5.8.4.2 4.2 Parallelism on a Multi-Core System

5.8.4.2.1 Mechanism: True Simultaneous Execution

1. **Core 0** runs **T1** and **T3** (switching between them).
2. **Core 1** runs **T2** and **T4** (switching between them).
3. **At any instant:**
 - Core 0 executes **one task** (e.g., T1).
 - Core 1 executes **another task** (e.g., T2).
 - **Result:** Two tasks run **truly in parallel**.

5.8.4.2.2 Key Observations

- **True parallelism:** Two tasks (one per core) execute **at the same time**.
- **Concurrency within cores:** Each core may still switch between tasks (e.g., Core 0 alternates T1 and T3).
- **Example:**
 - Time 0-0.1s: Core 0 runs T1; Core 1 runs T2 (**parallel**).
 - Time 0.1-0.2s: Core 0 runs T3; Core 1 runs T4 (**parallel**).
 - Time 0.2-0.3s: Core 0 switches back to T1; Core 1 switches back to T2.

5.8.5 5. Visual Representation of Concepts

5.8.5.1 5.1 Single-Core Concurrency (No Parallelism)

CPU 0: |----T1----|----T2----|----T3----|----T4----|----T1----|...

- **Interpretation:**
 - Only **one task** runs at a time.
 - **Concurrency:** Tasks take turns, creating the illusion of parallelism.

5.8.5.2 5.2 Dual-Core Parallelism + Concurrency

CPU 0: |----T1----|----T3----|----T1----|----T3----|...
CPU 1: |----T2----|----T4----|----T2----|----T4----|...

- **Interpretation:**
 - **Parallelism:** At any time, **two tasks** run (one per core).
 - **Concurrency:** Each core switches between tasks (e.g., CPU 0 alternates T1/T3).

5.8.6 6. Summary of Key Takeaways

1. Parallelism:

- **True simultaneity** (multiple tasks at once).
- Requires **multi-core/multi-processor**.
- **Faster execution** (tasks complete in less time).

2. Concurrency:

- **Illusion of simultaneity** (tasks progress together via switching).
- Works on **single-core** or multi-core.
- **No guaranteed speedup** (just perceived progress).

3. Relationship:

- All parallel systems are concurrent.
- Not all concurrent systems are parallel.

4. Real-World Analogy:

- **Parallelism**: Two chefs cooking two dishes **at the same time** (two stoves).
- **Concurrency**: One chef **switching rapidly** between two dishes (one stove).

5.9 Recording of Operating Systems Week 4 - Live Session on 26-04-04

5.9.1 Introduction to Threads

5.9.1.1 Recap of Process Scheduling

- **Long-Term Scheduler (Job Scheduler):**

- Acts as the **entry point** for any program.
- Decides whether a program is sent to the **ready queue**.
- If the long-term scheduler blocks a program, it **cannot execute** or use CPU resources.
- Controls the **degree of multiprogramming** (number of processes in memory).
- Example: If a program is **I/O-intensive**, the scheduler may **reject** it to prevent excessive interruptions.

5.9.2 Concept of Threads

5.9.2.1 Definition

- A **thread** is a **unit of execution** within a process.
- It is a **lightweight process** that runs **concurrently** with other threads in the same process.
- Threads enable **multitasking** on a **single processor** through **context switching**.

5.9.2.2 Key Characteristics

- **Concurrency vs. Parallelism:**

- **Concurrency**: Multiple tasks **appear** to run simultaneously due to rapid **context switching** (single processor).
- **Parallelism**: Multiple tasks **actually run simultaneously** (requires **multiple processors**).

- **Example:**

- Running a **presentation**, **speaking on Zoom**, **displaying system time**, and **printing a document** simultaneously on a single machine.

5.9.2.3 Why Use Threads?

1. Multitasking on a Single Process:

- Allows a process to perform **multiple operations simultaneously** (e.g., a web browser displaying images while fetching network data).

2. **Resource Efficiency:**
 - Threads **share memory and resources** of their parent process, reducing overhead.
3. **Responsiveness:**
 - If one thread is **blocked** (e.g., waiting for I/O), other threads can continue execution.
4. **Economic Resource Usage:**
 - Creating a new process requires **memory allocation**, while threads **reuse existing process memory**.
5. **Scalability:**
 - In **multiprocessor systems**, threads can run **in parallel** on different cores.

5.9.3 Difference Between Processes and Threads

Feature	Process	Thread
Weight	Heavy (resource-intensive)	Lightweight (shares process memory)
Memory Allocation	Requires separate memory	Shares process memory
Communication	Requires IPC (Inter-Process Communication)	Direct communication (shared memory)
Blocking Behavior	If one process blocks, others cannot run	If one thread blocks, others can run
Overhead	High (process creation/management)	Low (thread creation/management)
OS Interaction	Requires OS calls for switching	No OS interaction (handled internally)
Independence	Processes run independently	Threads coordinate within a process

5.9.3.1 Example Scenarios

- **Web Browser:**
 - One thread **displays images/text**, another **fetches data from the network**.
- **Web Server:**
 - A **single process** handles multiple client requests using **threads** (instead of spawning new processes).
- **Human Multitasking Analogy:**
 - Watching sports (main task) while eating (secondary task).
 - If an **interruption** (e.g., spicy food) occurs, the brain **switches focus** (context switching).

5.9.4 Thread Models

5.9.4.1 1. Many-to-One Model

- **Description:**
 - **Multiple user-level threads** map to **one kernel thread**.
- **Pros:**
 - Simple to implement.
- **Cons:**
 - **Blocking issue:** If one thread makes a **blocking system call**, all threads **halt** (since only one kernel thread exists).
 - **No parallelism:** Cannot utilize multiprocessor systems.

5.9.4.2 2. One-to-One Model

- **Description:**
 - Each user thread maps to **one kernel thread**.

- **Pros:**
 - **True parallelism:** Threads can run on **different processors**.
 - **No blocking issue:** If one thread blocks, others can continue.
- **Cons:**
 - **Overhead:** Creating too many kernel threads can **degrade performance**.
 - **System limits:** Most OS restrict the number of threads.

5.9.4.3 3. Many-to-Many Model

- **Description:**
 - **Multiple user threads** map to a **smaller/equal number of kernel threads**.
- **Pros:**
 - **Flexibility:** Kernel can schedule threads **independently**.
 - **Scalability:** Can utilize **multiprocessor systems**.
- **Cons:**
 - **Complexity:** Requires **sophisticated thread management**.

5.9.5 Thread Creation and Management

5.9.5.1 Thread Creation (POSIX Threads - pthread.h)

- **Function:** pthread_create()
 - **Syntax:**

```
pthread_create(&thread_id, NULL, function_name, arguments);
```
 - **Parameters:**
 1. thread_id: Identifier for the new thread.
 2. NULL: Default thread attributes.
 3. function_name: The **code** the thread will execute.
 4. arguments: Optional arguments passed to the function.
- **Example:**

```
pthread_t t1, t2;
pthread_create(&t1, NULL, task1, NULL);
pthread_create(&t2, NULL, task2, NULL);
```

 - task1 and task2 are **units of execution** running concurrently.

5.9.5.2 Thread Joining (pthread_join)

- **Purpose:** Ensures the **main thread waits** for other threads to complete.
- **Syntax:**

```
pthread_join(thread_id, NULL);
```
- **Analogy:**
 - **Kitchen Example:**
 - * **Process** = Kitchen (shared space).
 - * **Threads** = Chefs (workers in the kitchen).
 - * pthread_create = Assigning tasks to chefs.

* `pthread_join` = Waiting for all chefs to finish before serving food.

5.9.5.3 Demo: Threads vs. Non-Threaded Execution

- **Non-Threaded Program:**
 - Tasks run **sequentially** (e.g., `task1` completes before `task2` starts).
 - **Problem:** Poor responsiveness (e.g., a UI freezes during a long operation).
- **Threaded Program:**
 - Tasks run **concurrently** (e.g., `task1` and `task2` alternate via context switching).
 - **Output Example:**
Task1: Counter 1
Task2: Counter 1
Task1: Counter 2
Task2: Counter 2
 - **Performance Impact:**
 - * Threaded programs may take **slightly longer** due to **overhead** (thread creation/management).
 - * Example: Non-threaded = **5.02s**, Threaded = **5.29s**.

5.9.6 Benefits of Multithreading

1. **Responsiveness:**
 - Interactive applications remain **responsive** even if one thread is blocked.
 - Example: A UI thread continues while a background thread performs a **time-consuming task**.
2. **Resource Sharing:**
 - Threads **share memory/data** of their process, reducing **memory allocation overhead**.
3. **Economical:**
 - Creating threads is **cheaper** than creating processes (no need for separate memory spaces).
4. **Scalability:**
 - On **multiprocessor systems**, threads can run **in parallel**, improving throughput.

5.9.7 Overhead of Threading

- **Thread Creation:**
 - Requires **additional system resources** (e.g., stack allocation).
- **Thread Management:**
 - **Scheduling:** Deciding which thread runs next.
 - **Priority Handling:** Managing thread priorities.
 - **Synchronization:** Preventing **race conditions** (e.g., mutual exclusion).
- **Performance Impact:**
 - Threaded programs may run **slower** due to **context switching overhead**.
 - **Rule of Thumb:** Use threads **only when necessary** (e.g., for concurrency, not for simple sequential tasks).

5.9.8 Key Takeaways

1. **Threads enable concurrency** on a single processor.
2. **Processes are heavyweight**; threads are **lightweight**.
3. **Thread models** (Many-to-One, One-to-One, Many-to-Many) trade off **complexity vs. performance**.
4. `pthread_create` and `pthread_join` are essential for thread management.
5. **Multithreading improves responsiveness and resource usage** but introduces **overhead**.
6. **Not all programs need threads**-use them **judiciously** based on requirements.

5.10 Synchronous vs Asynchronous Multithreading

5.10.1 1. Introduction to Multithreading Models

- Multithreading allows a program to execute multiple threads concurrently.
- Two primary models of multithreading:
 - **Asynchronous Multithreading**
 - **Synchronous Multithreading**
- These models differ in how parent and child threads interact during execution.

5.10.2 2. Asynchronous Multithreading

5.10.2.1 2.1 Definition

- A multithreading model where a **parent thread** creates one or more **child threads**.
- Upon creation, the **parent thread resumes execution immediately** without waiting for child threads.
- **No synchronization** between parent and child threads; they execute **independently**.

5.10.2.2 2.2 Key Characteristics

- **Concurrent Execution:**
 - Parent and child threads **run simultaneously** if sufficient processing cores are available.
 - No inherent dependency between threads.
- **No Waiting Mechanism:**
 - The parent thread **does not wait** for child threads to complete.
 - Child threads may terminate at any time, and the parent remains unaware of their termination.
- **Limited Data Sharing:**
 - Due to independent execution, **minimal data sharing** occurs between threads.
 - Threads operate with **less coordination**, reducing the need for shared resources.

5.10.2.3 2.3 Execution Flow

1. Parent thread **spawns** child threads.
2. Parent **continues execution** without pausing.
3. Child threads execute **in parallel** (if hardware permits).
4. Parent **does not track** child thread termination.
5. No explicit **joining** of child threads with the parent.

5.10.2.4 2.4 Use Cases

- Suitable for **independent tasks** where threads do not need to synchronize.
- Example: Background tasks (e.g., logging, monitoring) that do not require parent thread awareness.

5.10.3 3. Synchronous Multithreading

5.10.3.1 3.1 Definition

- A multithreading model where a **parent thread** creates child threads but **waits for their completion** before resuming execution.
- Follows a **fork-join** strategy:
 - **Fork:** Parent creates child threads.
 - **Join:** Parent waits for all child threads to terminate and **rejoins** them.

5.10.3.2 3.2 Key Characteristics

- **Parent Thread Waiting:**
 - After spawning child threads, the parent **enters a waiting state**.
 - Parent **cannot resume** until **all child threads** complete execution.
- **Explicit Joining Mechanism:**
 - Child threads **must join** the parent upon completion (e.g., using `pthread_join` in POSIX threads).
 - Ensures **synchronization** between parent and children.
- **Increased Data Sharing:**
 - Due to coordinated execution, threads **share more data** compared to asynchronous multithreading.
 - Useful for **collaborative tasks** requiring shared resources.

5.10.3.3 3.3 Execution Flow

1. Parent thread **creates** child threads.
2. Parent **pauses execution** and waits (`pthread_join`).
3. Child threads execute **concurrently** (if hardware permits).
4. Upon completion, each child thread **joins** the parent.
5. Parent **resumes execution** only after **all child threads** have terminated.

5.10.3.4 3.4 Use Cases

- Suitable for **dependent tasks** where the parent requires results from child threads.
- Example:
 - Parallel computation (e.g., dividing a large task into subtasks).
 - Workflows where the parent must aggregate results from children.

5.10.4 4. Comparative Analysis

Feature	Asynchronous Multithreading	Synchronous Multithreading
Parent Thread Behavior	Continues execution immediately after spawning threads.	Waits for all child threads to complete before resuming.
Thread Independence	High (threads execute without coordination).	Low (parent depends on child completion).
Data Sharing	Minimal (less coordination needed).	Higher (threads often share data).
Joining Mechanism	No explicit joining.	Explicit joining (e.g., <code>pthread_join</code>).
Concurrency	Parent and children execute in parallel.	Only children execute concurrently; parent waits.
Use Case	Independent background tasks.	Collaborative or dependent tasks.

5.10.5 5. Key Terms and Definitions

- **Parent Thread:** The thread that creates one or more child threads.
- **Child Thread:** A thread spawned by a parent thread.
- **Fork-Join Model:**
 - **Fork:** Creation of child threads.
 - **Join:** Parent thread waiting for child threads to complete.
- **pthread_join:** A POSIX function used to make a parent thread wait for a child thread's termination.
- **Concurrent Execution:** Multiple threads running in overlapping time periods (not necessarily simultaneously).

- **Parallel Execution:** Multiple threads running **simultaneously** on different CPU cores.
- **Thread Independence:** The degree to which threads operate without relying on each other.

5.10.6 6. Practical Implications

5.10.6.1 6.1 Performance Considerations

- **Asynchronous:**
 - Better for **I/O-bound** or **non-blocking** tasks.
 - Reduces **idle time** for the parent thread.
- **Synchronous:**
 - Better for **CPU-bound** tasks requiring **coordination**.
 - May introduce **bottlenecks** if child threads take too long.

5.10.6.2 6.2 Programming Complexity

- **Asynchronous:**
 - Simpler to implement (no need for joining logic).
 - Risk of **race conditions** if threads share data without synchronization.
- **Synchronous:**
 - More complex due to **joining and synchronization**.
 - Easier to manage **shared resources** safely.

5.10.6.3 6.3 Resource Utilization

- **Asynchronous:**
 - **Higher CPU utilization** (parent and children run concurrently).
 - May lead to **resource contention** if too many threads run simultaneously.
- **Synchronous:**
 - **Controlled resource usage** (parent waits, reducing active threads).
 - May **underutilize CPU** if parent waits excessively.

5.10.7 7. Example Scenarios

5.10.7.1 7.1 Asynchronous Multithreading Example

- **Scenario:** A web server handling multiple client requests.
 - Parent thread spawns a child thread for each request.
 - Parent continues accepting new requests **without waiting** for responses.
 - Child threads process requests **independently**.

5.10.7.2 7.2 Synchronous Multithreading Example

- **Scenario:** A data processing pipeline.
 - Parent thread divides a large dataset into chunks.
 - Spawns child threads to process each chunk.
 - Parent **waits** for all chunks to be processed before combining results.

5.10.8 8. Summary of Key Takeaways

1. **Asynchronous Multithreading:**
 - Parent and child threads execute **independently**.
 - No waiting; **higher concurrency** but **less data sharing**.

- Ideal for **non-dependent tasks**.
2. **Synchronous Multithreading:**
 - Parent **waits** for child threads to complete (**fork-join model**).
 - **More data sharing** but **lower concurrency** for the parent.
 - Ideal for **dependent or collaborative tasks**.
 3. **Choice Depends on:**
 - Task dependencies.
 - Need for synchronization.
 - Performance and resource constraints.

5.11 Thread Cancellation

5.11.1 Introduction to Thread Cancellation

- **Definition:** Thread cancellation refers to the process of terminating a specific thread before it has completed its execution.
- **Target Thread:** The thread that is intended to be canceled or terminated is referred to as the **target thread**.
- **Purpose:** Used to manage thread execution dynamically, especially in scenarios where a thread's continued operation is no longer necessary or desirable.

5.11.2 Thread Cancellation in POSIX Pthreads

- **Function:** The `pthread_cancel` function is used to request the cancellation of a target thread.
 - **Signature:**

```
int pthread_cancel(pthread_t thread);
```
 - **Parameters:**
 - * `thread`: A variable of type `pthread_t` representing the identifier of the target thread.
 - **Return Type:** `int` (returns 0 on success, or an error code on failure).
- **Mechanism:**
 - When `pthread_cancel` is invoked with a thread identifier, a **cancellation request** is sent to the target thread.
 - The response of the target thread to this request depends on its **cancellation type**.

5.11.3 Types of Thread Cancellation

Thread cancellation can be classified into two primary types, each with distinct behaviors and implications:

5.11.3.1 1. Asynchronous Cancellation

- **Definition:** The target thread is terminated **immediately** upon receiving a cancellation request.
- **Behavior:**
 - The target thread does not perform any cleanup or finalization before termination.
 - Resources allocated to the thread (e.g., memory, file handles) may or may not be reclaimed by the operating system.
 - If the thread was in the process of updating **shared data** (e.g., a database, table, or file), the data may be left in an **inconsistent or corrupted state**.
- **Problems:**
 - **Resource Leaks:** Some resources may not be released, preventing their allocation to other threads.
 - **Data Corruption:** Shared data may remain in an unstable state, leading to system-wide inconsistencies.
- **Recommendation:**

- Asynchronous cancellation is **not recommended** in most scenarios due to the risks of resource leaks and data corruption.
- While supported in the Pthreads library, its use is discouraged in favor of safer alternatives.

5.11.3.2 2. Deferred Cancellation

- **Definition:** The target thread is **not terminated immediately** upon receiving a cancellation request. Instead, it checks for a **safe point** (cancellation point) before terminating.
- **Behavior:**
 - The target thread continues execution until it reaches a **cancellation point**, a predefined location in the code where it is safe to terminate.
 - Upon reaching a cancellation point, the thread checks for pending cancellation requests.
 - If a cancellation request is pending, the thread invokes a **cleanup handler** to:
 - * Release allocated resources (e.g., memory, file descriptors).
 - * Complete critical operations (e.g., finalizing updates to shared data).
 - * Ensure the system remains in a consistent state.
 - After cleanup, the thread terminates **orderly**.
- **Advantages:**
 - Prevents resource leaks by ensuring proper release of allocated resources.
 - Avoids data corruption by allowing the thread to complete critical operations.
 - Provides a **controlled and predictable** termination process.
- **Default in Pthreads:**
 - Deferred cancellation is the **default cancellation type** in the Pthreads library due to its safety and reliability.

5.11.4 Cancellation Points

- **Definition:** A **cancellation point** is a specific location in a thread's execution where it is safe to respond to a cancellation request.
- **Purpose:**
 - Ensures the thread can terminate without causing system instability.
 - Allows the thread to perform necessary cleanup before termination.
- **Creating Cancellation Points:**
 - In Pthreads, the `pthread_testcancel` function can be used to **explicitly create a cancellation point**.
 - * **Signature:**

```
void pthread_testcancel(void);
```
 - * **Parameters:** None.
 - * **Return Type:** void.
 - * **Behavior:**
 - When invoked, the function checks for pending cancellation requests.
 - If a request is pending, the thread performs cleanup (if any) and terminates.
 - If no request is pending, the thread continues execution.
- **Examples of Cancellation Points:**
 - Standard library functions (e.g., `malloc`, `free`, `open`, `read`, `write`) often include implicit cancellation points.
 - Custom cancellation points can be added using `pthread_testcancel` in critical sections of code.

5.11.5 Summary of Key Concepts

1. **Thread Cancellation:** Terminating a thread before its natural completion.

2. **Target Thread:** The thread selected for cancellation.
3. **pthread_cancel:** Function to request cancellation of a target thread.
4. **Cancellation Types:**
 - **Asynchronous:** Immediate termination (risky, not recommended).
 - **Deferred:** Termination at safe cancellation points (recommended).
5. **Cancellation Points:** Safe locations in code where a thread can respond to cancellation requests.
6. **pthread_testcancel:** Function to explicitly create a cancellation point.
7. **Cleanup Handlers:** Routines executed to release resources and ensure data consistency before termination.

5.11.6 Best Practices

- **Avoid Asynchronous Cancellation:** Use deferred cancellation to prevent resource leaks and data corruption.
- **Define Cancellation Points:** Strategically place cancellation points in threads to ensure safe termination.
- **Use Cleanup Handlers:** Implement cleanup handlers to manage resources and shared data properly.
- **Default to Deferred Cancellation:** Leverage the default deferred cancellation in Pthreads for safer thread management.

5.12 Thread functions

5.12.1 Introduction to Thread Functions

- This lecture covers **thread functions** provided by the **pthread (POSIX threads) library**.
- Focuses on:
 - Function signatures (arguments and return types).
 - Purpose and usage of each function.
 - Error handling mechanisms.

5.12.2 1. Thread Attributes Initialization: pthread_attr_init

5.12.2.1 Purpose

- Initializes a **thread attributes object** with **default values**.
- Used to configure thread properties (e.g., stack size, scheduling policy) before thread creation.

```
int pthread_attr_init(pthread_attr_t *attr);
```

5.12.2.2 Function Signature

- **Argument:**
 - **attr:** Pointer to a **pthread_attr_t** object (thread attributes structure).
- **Return Type:** **int** (integer).
 - **Success:** Returns **0**.
 - **Failure:** Returns a **non-zero error number** (indicates the specific error).

5.12.2.3 Key Points

- The attributes object (**attr**) must be **reinitialized** if destroyed (e.g., via **pthread_attr_destroy**).
- Default values are assigned to all attributes (e.g., default stack size, default scheduling policy).
- **Error Handling:** Non-zero return values correspond to specific error codes (e.g., **EINVAL** for invalid arguments).

5.12.3 2. Thread Attributes Destruction: pthread_attr_destroy

5.12.3.1 Purpose

- **Deallocates resources** associated with a thread attributes object.
- Must be called when the attributes object is **no longer needed**.

```
int pthread_attr_destroy(pthread_attr_t *attr);
```

5.12.3.2 Function Signature

- **Argument:**
 - attr: Pointer to the pthread_attr_t object to be destroyed.
- **Return Type:** int.
 - **Success:** Returns 0.
 - **Failure:** Returns a **non-zero error number**.

5.12.3.3 Key Points

- **Safety Constraint:** **Never destroy** an attributes object while it is **still in use** (e.g., being referenced by a thread).
- After destruction, the object **cannot be reused** unless reinitialized with pthread_attr_init.
- **Error Handling:** Non-zero return values indicate specific errors (e.g., EINVAL if attr is invalid).

5.12.4 3. Thread Creation: pthread_create

5.12.4.1 Purpose

- **Creates a new thread** with specified attributes and execution routine.

```
int pthread_create(
    pthread_t *tid,
    const pthread_attr_t *attr,
    void *(*start_routine)(void *),
    void *arg
);
```

5.12.4.2 Function Signature

- **Arguments:**
 1. tid: Pointer to a pthread_t variable (stores the **thread ID** of the newly created thread).
 2. attr: Pointer to a pthread_attr_t object (specifies thread attributes; NULL for defaults).
 - **const qualifier:** Ensures the attributes object is **not modified** during thread creation.
 3. start_routine: Pointer to the **function** the thread will execute.
 - **Signature:** Must match void *(*function)(void *).
 - **Why void*?**
 - * Allows **generic argument/return types** (e.g., threads may pass/return int*, float*, custom structs).
 4. arg: Argument passed to start_routine (type void* for generality).
- **Return Type:** int.
 - **Success:** Returns 0.

- **Failure:** Returns a **non-zero error number** (e.g., `EAGAIN` for resource limits, `EINVAL` for invalid arguments).

5.12.4.3 Key Points

- The new thread **starts execution** at `start_routine` with `arg` as its argument.
- **Thread ID (tid)** is assigned by the system and stored in the provided `pthread_t` variable.
- **Error Handling:** Check return value to confirm successful thread creation.

5.12.5 4. Thread Termination: `pthread_exit`

5.12.5.1 Purpose

- **Terminates the calling thread** and optionally provides an **exit status**.

```
void pthread_exit(void *ptr);
```

5.12.5.2 Function Signature

- **Argument:**
 - `ptr`: Pointer to the **exit status** (type `void*`).
 - * Can be `NULL` if no status is needed.
 - * Used by other threads (e.g., via `pthread_join`) to retrieve the termination status.
- **Return Type:** `void` (no return value).

5.12.5.3 Key Points

- **Implicit Termination:** A thread also terminates if its `start_routine` returns (equivalent to calling `pthread_exit` with the return value).
- **Exit Status:**
 - Useful for **synchronization** (e.g., a waiting thread may need the result of the terminating thread).
 - Example: If a thread computes a result, it can pass a pointer to that result via `ptr`.

5.12.6 5. Thread Joining: `pthread_join`

5.12.6.1 Purpose

- **Blocks the calling thread** until a **target thread** terminates.
- Optionally retrieves the **exit status** of the target thread.

```
int pthread_join(pthread_t thread, void **retval);
```

5.12.6.2 Function Signature

- **Arguments:**
 1. `thread`: `pthread_t` identifier of the **target thread** to wait for.
 2. `retval`: Double pointer (`void**`) to store the **exit status** of the target thread.
 - If `NULL`, the exit status is **ignored**.
 - If non-`NULL`, the status is copied to the location pointed to by `retval`.
- **Return Type:** `int`.
 - **Success:** Returns `0`.

- **Failure:** Returns a **non-zero error number** (e.g., EINVAL for invalid thread ID, ESRCH if the thread is not joinable).

5.12.6.3 Key Points

- **Blocking Behavior:** The calling thread is **suspended** until the target thread terminates.
- **Exit Status Retrieval:**
 - If the target thread called `pthread_exit(ptr)`, `*retval` will point to `ptr`.
 - If the target thread returned from `start_routine`, `*retval` will point to the return value.
- **Use Cases:**
 - **Synchronization:** Ensure a thread completes before proceeding.
 - **Resource Cleanup:** Reclaim resources (e.g., memory) allocated by the target thread.

5.12.7 6. Thread ID Comparison: pthread_equal

5.12.7.1 Purpose

- **Compares two thread IDs** to check if they refer to the same thread.

```
int pthread_equal(pthread_t tid1, pthread_t tid2);
```

5.12.7.2 Function Signature

- **Arguments:**
 - `tid1, tid2`: Thread IDs (`pthread_t`) to compare.
- **Return Type:** `int`.
 - **Equal:** Returns a **non-zero value**.
 - **Not Equal:** Returns `0`.

5.12.7.3 Key Points

- **Why Not Direct Comparison?**
 - `pthread_t` may be an **opaque type** (e.g., a struct or integer), so direct comparison (e.g., `tid1 == tid2`) is **not portable**.
- **Use Cases:**
 - Check if a thread is **referring to itself** (e.g., `pthread_equal(tid, pthread_self())`).
 - Validate thread IDs in **synchronization logic**.

5.12.8 Summary of Key Concepts

Function	Purpose	Return Type	Key Arguments
<code>pthread_attr_t</code>	Initialize thread attributes with defaults.	<code>int</code>	<code>pthread_attr_t *attr</code>
<code>pthread_attr_t</code>	Destroy a thread attributes object.	<code>int</code>	<code>pthread_attr_t *attr</code>
<code>pthread_create</code>	Create a new thread with specified attributes and routine.	<code>int</code>	<code>pthread_t *tid,</code> <code>pthread_attr_t *attr,</code> <code>void</code> <code>*(start_routine)(void *, void *arg)</code>

Function	Purpose	Return Type	Key Arguments
<code>pthread_exit</code>	Terminate the calling thread with an optional exit status.	<code>void</code>	<code>void *ptr</code>
<code>pthread_join</code>	Wait for a thread to terminate and retrieve its exit status.	<code>int</code>	<code>pthread_t thread, void **retval</code>
<code>pthread_equal</code>	Compare two thread IDs for equality.	<code>int</code>	<code>pthread_t tid1, pthread_t tid2</code>

5.12.8.1 Error Handling

- All functions (except `pthread_exit`) return:
 - `0` on **success**.
 - **Non-zero error number** on failure (check `errno` or pthread-specific errors like `EINVAL`, `EAGAIN`).

5.12.8.2 General Notes

- **Thread Safety:** Pthreads functions are **thread-safe** (can be called from any thread).
- **Portability:** Pthreads is a **POSIX standard**; code using these functions is portable across Unix-like systems.
- **Memory Management:** Attributes objects (`pthread_attr_t`) must be **properly initialized and destroyed** to avoid leaks.

5.13 Thread related data structures

5.13.1 1. Introduction to Thread Libraries

5.13.1.1 1.1 Definition of a Thread Library

- A **thread library** provides an **Application Programming Interface (API)** for:
 - **Creation** of threads.
 - **Management** of threads.
- The API consists of:
 - **Functions** (for thread operations).
 - **Data structures** (to represent thread properties).

5.13.1.2 1.2 Role of a Thread Library

- Enables **application programmers** to:
 - Spawn new threads.
 - Control thread execution (e.g., synchronization, scheduling).
 - Manage thread attributes (e.g., stack size, detach state).

5.13.2 2. Implementation Models of Thread Libraries

Thread libraries can be implemented in **two primary ways**, differing in their interaction with the **operating system (OS) kernel**:

5.13.2.1 2.1 User-Level Thread Library

- **Location:** Entirely in **user space** (no kernel involvement).
- **Components:**
 - Code for thread operations (e.g., creation, termination).

- Associated **data structures** (e.g., thread control blocks).
- **API Calls:**
 - Treated as **local procedure calls** (no system calls).
 - Example: If a thread invokes a library function (e.g., `pthread_create`), the call is resolved **without kernel intervention**.
- **Key Characteristic:**
 - **No OS support required** (library is self-contained in user space).

5.13.2.2 2.2 Kernel-Level Thread Library

- **Location:** Entirely in **kernel space** (requires OS support).
- **Components:**
 - Thread management code **inside the OS kernel**.
 - Kernel-level data structures (e.g., kernel thread control blocks).
- **API Calls:**
 - Require **system calls** (e.g., a thread invoking `pthread_create` triggers a kernel operation).
- **Key Characteristic:**
 - **Direct OS involvement** in thread management.
 - Threads execute in **kernel mode**.

5.13.3 3. Pthreads (POSIX Threads) Library

5.13.3.1 3.1 Overview

- **Definition:** A **standardized thread API** defined by **POSIX (Portable Operating System Interface)**.
- **Purpose:**
 - Provides a **uniform interface** for thread creation and synchronization.
 - Ensures **portability** across POSIX-compliant systems.
- **Nature:**
 - **Specification-only:** Defines **behavior** (what threads should do) but **not implementation** (how they do it).
 - Implementations may vary across operating systems.

5.13.3.2 3.2 Compatible Operating Systems POSIX-compliant OSes supporting Pthreads include: - **Solaris** - **Linux** - **macOS**

5.13.3.3 3.3 Key Distinction: Specification vs. Implementation

- **POSIX Standard:**
 - Dictates **thread behavior** (e.g., how threads respond to synchronization calls).
- **Implementation:**
 - Left to **OS developers** (e.g., Linux and macOS may implement Pthreads differently).
 - Must adhere to the **POSIX behavior specification**.

5.13.4 4. Pthreads Data Structures

Pthreads defines **two primary data structures** to represent threads and their attributes:

5.13.4.1 4.1 pthread_t (Thread Identifier)

5.13.4.1.1 4.1.1 Definition

- A **data type** used to **uniquely identify a thread**.
- Defined in the header file: `pthread.h`.

5.13.4.1.2 4.1.2 Key Properties

- **Opaque Data Type:**
 - Must **not** be treated as a primitive type (e.g., `int`, `unsigned long`).
 - **Reason:** Ensures **portability** across POSIX-compliant systems.
 - * Example: One OS may implement `pthread_t` as an `unsigned int`, another as a `struct`.
- **Purpose:**
 - Acts as a **handle** for thread operations (e.g., joining, canceling).

5.13.4.1.3 4.1.3 Why Not a Primitive Type?

- **Portability Risk:**
 - If a program assumes `pthread_t` is an `int`, it may fail on systems where `pthread_t` is a `struct`.
 - **POSIX compliance** requires treating it as an **abstract type**.

5.13.4.1.4 4.1.4 Implementation Variations

- Possible underlying types:
 - `unsigned int`
 - `unsigned long`
 - `struct` (containing thread metadata)

5.13.4.2 4.2 pthread_attr_t (Thread Attributes Object)

5.13.4.2.1 4.2.1 Definition

- A **data structure** representing a **set of thread attributes**.
- Defined in `pthread.h`.

5.13.4.2.2 4.2.2 Attributes Covered

Attribute	Description
Detach State	Whether the thread is joinable or detached (cannot be joined).
Scope	Process-wide (competes with all threads) or system-wide (kernel-managed).
Stack Size	Size of the thread's call stack (in bytes).
Stack Address	Starting address of the thread's stack (if custom stack is used).
Scheduling Policy	Algorithm for thread scheduling (e.g., FIFO , Round Robin , Other).
Scheduling Priority	Priority level for thread execution (higher = more CPU time).

5.13.4.2.3 4.2.3 Purpose

- Configures **thread behavior** before creation (via `pthread_create`).
- Example: Setting a thread to **detached** mode prevents resource leaks if not joined.

5.13.5 5. Classification of Pthreads Data Structures

5.13.5.1 5.1 Why Are They Called “Data Structures”?

- `pthread_t`:
 - Though often a **single value** (e.g., an ID), it is **opaque** (implementation-defined).
 - Encapsulates **thread identity** (hence treated as a structure in abstract terms).
- `pthread_attr_t`:
 - Clearly a **structure** holding multiple attributes.

5.13.5.2 5.2 Unified Terminology

- Both are grouped under “**Pthreads data structures**” because:
 - They **store thread-related information**.
 - They **abstract thread properties** for portability.

5.13.6 6. Summary of Key Concepts

1. **Thread Library**:
 - Provides APIs for thread creation/management.
 - Can be **user-level** (no kernel) or **kernel-level** (OS-supported).
2. **Pthreads**:
 - POSIX-standardized thread API.
 - **Behavior is specified**; implementation varies by OS.
3. **Data Structures**:
 - `pthread_t`: Thread identifier (opaque, not primitive).
 - `pthread_attr_t`: Thread attributes (detach state, stack size, etc.).
4. **Portability**:
 - Treating `pthread_t` as an abstract type ensures **cross-platform compatibility**.

5.14 Types of Parallelism

5.14.1 Introduction

- Parallelism refers to the simultaneous execution of multiple tasks or operations to improve computational efficiency.
- Two primary types of parallelism exist in computing:
 1. **Data Parallelism**
 2. **Task Parallelism**
- These types differ in how workloads are distributed across processors or cores.

5.14.2 Data Parallelism

5.14.2.1 Definition

- **Data parallelism** involves distributing **subsets of the same data** across different processors or processing cores.
- Each processor/core executes the **same task or operation** on its assigned data subset.
- Key characteristic: **Same operation, different data segments**.

5.14.2.2 Mechanism

- A large dataset is partitioned into smaller segments.

- Each thread/processor operates on a distinct segment of the data.
- Results from individual threads are combined to produce the final output.

5.14.2.3 Example: Maximum Value Computation

- **Scenario:** An array (`list`) contains 1,000 numbers. The goal is to find the maximum value in the array using a multi-threaded program with four threads (T1, T2, T3, T4).

5.14.2.3.1 Data Partitioning

- The array is divided into four equal segments (assuming zero-based indexing):
 1. **Thread T1:** Computes the maximum value in the range `list[0]` to `list[249]`.
 2. **Thread T2:** Computes the maximum value in the range `list[250]` to `list[499]`.
 3. **Thread T3:** Computes the maximum value in the range `list[500]` to `list[749]`.
 4. **Thread T4:** Computes the maximum value in the range `list[750]` to `list[999]`.

5.14.2.3.2 Execution

- Each thread independently computes the maximum value for its assigned segment.
- The threads operate **concurrently** on different data subsets.

5.14.2.3.3 Result Aggregation

- After all threads complete their computations, the **final maximum value** is determined by comparing the four intermediate results (one from each thread).
- Example: If T1 reports 50, T2 reports 75, T3 reports 60, and T4 reports 80, the overall maximum is 80.

5.14.2.4 Key Observations

- **Uniform Task:** All threads perform the same operation (finding the maximum).
- **Distinct Data:** Each thread works on a unique subset of the dataset.
- **Efficiency:** Parallelism reduces computation time by leveraging multiple processors/cores.

5.14.3 Task Parallelism

5.14.3.1 Definition

- **Task parallelism** involves distributing **different tasks or operations** across multiple processors or processing cores.
- Each thread/processor executes a **unique task**, which may operate on the same or different datasets.
- Key characteristic: **Different operations, same or different data.**

5.14.3.2 Mechanism

- Multiple distinct tasks are assigned to separate threads.
- Threads may access shared data or independent datasets.
- Results are combined based on the requirements of the application.

5.14.3.3 Example: Statistical Computations on an Array

- **Scenario:** An array (`list`) contains 1,000 numbers. Four statistical operations are to be performed concurrently using a multi-threaded program with four threads (T1, T2, T3, T4).

5.14.3.3.1 Task Assignment

- Each thread is assigned a unique operation:
 1. **Thread T1**: Computes the **maximum** value in the array.
 2. **Thread T2**: Computes the **minimum** value in the array.
 3. **Thread T3**: Computes the **mean (average)** of all numbers in the array.
 4. **Thread T4**: Computes the **median** of all numbers in the array.

5.14.3.3.2 Execution

- All threads operate on the **entire dataset** (`list[0]` to `list[999]`).
- Each thread performs a **distinct computation**:
 - T1 iterates through the array to find the maximum.
 - T2 iterates through the array to find the minimum.
 - T3 sums all elements and divides by the count (1,000) to compute the mean.
 - T4 sorts the array (or uses a selection algorithm) to find the median.

5.14.3.3.3 Result Aggregation

- Each thread produces an independent result:
 - Maximum value (from T1).
 - Minimum value (from T2).
 - Mean value (from T3).
 - Median value (from T4).
- No further aggregation is required unless the application demands a combined analysis (e.g., range = max - min).

5.14.3.4 Key Observations

- **Diverse Tasks**: Each thread performs a unique operation.
- **Shared Data**: All threads may access the same dataset (though this is not a requirement; data could also differ).
- **Flexibility**: Task parallelism is suitable for applications requiring multiple independent computations.

5.14.4 Comparison of Data Parallelism and Task Parallelism

Aspect	Data Parallelism	Task Parallelism
Task Uniformity	Same task executed on different data subsets.	Different tasks executed on same or different data.
Data Handling	Data is partitioned into subsets.	Data may be shared or independent.
Example Use Case	Finding max/min in a large array.	Performing multiple statistics (max, min, mean, median) on an array.
Concurrency Focus	Divides data for parallel processing.	Divides tasks for parallel execution.
Result Aggregation	Intermediate results are combined (e.g., comparing max values).	Results are independent (e.g., max, min, mean, median).

5.14.5 Practical Implications

- **Data Parallelism** is ideal for:
 - Problems where the same operation is applied to large datasets (e.g., image processing, matrix operations).

- Scenarios where data can be easily partitioned (e.g., dividing an array into chunks).
- **Task Parallelism** is ideal for:
 - Problems requiring multiple distinct operations (e.g., scientific simulations, multi-step data analysis).
 - Applications where tasks are independent and can run concurrently (e.g., web servers handling multiple requests).

5.14.6 Summary

- **Data Parallelism:** Focuses on distributing data across processors to perform the same task concurrently.
- **Task Parallelism:** Focuses on distributing different tasks across processors, which may operate on shared or independent data.
- Both types of parallelism aim to improve performance by leveraging multi-core architectures, but they are applied in different contexts based on the nature of the problem.

5.15 User level threads & Kernel level threads

5.15.1 1. Introduction

This lecture explores the two primary types of threads in operating systems: - **User-level threads (ULT)** - **Kernel-level threads (KLT)**

The discussion covers their definitions, management, advantages, limitations, and key differences.

5.15.2 2. User-Level Threads (ULT)

5.15.2.1 2.1 Definition

- **User-level threads** are threads created and managed entirely by a **threads library** in **user space**.
- The **kernel is unaware** of their existence-it treats a multi-threaded process using ULTs as a **single-threaded process**.
- All code and data structures for ULTs reside **exclusively in user space**-no part of the library exists in kernel space.

5.15.2.2 2.2 Key Characteristics

1. No Kernel Support

- The threads library operates **independently of the kernel**.
- The kernel does **not recognize or schedule** individual ULTs.

2. Blocking Behavior

- If **one ULT performs a blocking operation** (e.g., a blocking system call), the **entire process blocks**.
 - Example: If a thread calls `read()` on a file and the operation blocks, all other ULTs in the process **halt** because the kernel sees only one thread.
- This is a **major limitation** of ULTs in terms of concurrency.

3. Context Switching

- **No kernel mode privileges required** for thread context switching.
- Switching between ULTs occurs **entirely in user space**, making it **faster** than kernel-level context switching.

4. Function Calls & API Invocations

- When a ULT invokes a function from the threads library (e.g., `pthread_create()`), it is treated as a **local function call** in user space.
- **No transition to kernel space** occurs-execution remains in user mode.
- Example: Calling `pthread_mutex_lock()` does not trigger a system call.

5. Thread Libraries Supporting ULTs Common user-level thread libraries include:

- **POSIX Threads (Pthreads)** - Standard in Unix-like systems.
- **Windows Threads** - Used in Windows applications.
- **Java Threads** - Managed by the Java Virtual Machine (JVM).

5.15.2.3 2.3 Advantages of User-Level Threads

- **Fast Context Switching:** No kernel involvement -> lower overhead.
- **Portability:** Thread management is handled by user-space libraries, making programs **more portable** across different OS kernels.
- **Flexibility:** Programmers can implement **custom scheduling policies** (e.g., priority-based scheduling) without kernel constraints.

5.15.2.4 2.4 Disadvantages of User-Level Threads

- **No True Parallelism:** Since the kernel sees only one thread, ULTs **cannot leverage multi-core processors** effectively.
- **Blocking Problem:** A single blocking system call **stalls the entire process**.
- **Limited Kernel Services:** Cannot directly use kernel-provided synchronization primitives (e.g., kernel-level semaphores).

5.15.3 3. Kernel-Level Threads (KLT)

5.15.3.1 3.1 Definition

- **Kernel-level threads** are threads **recognized and managed by the kernel**.
- They are created and controlled via **system calls** provided by the OS.
- The **kernel schedules and executes KLT** independently, allowing **true parallelism** on multi-core systems.

5.15.3.2 3.2 Key Characteristics

1. Kernel Recognition & Management

- The kernel **maintains thread control blocks (TCBs)** for each KLT.
- Thread operations (creation, termination, scheduling) require **system calls**.

2. Multi-Core Utilization

- Since the kernel is aware of multiple threads, it can **schedule them on different CPU cores**, enabling **true parallel execution**.

3. Blocking Behavior

- If **one KLT performs a blocking operation** (e.g., I/O wait), **other KLTs in the same process can continue executing**.
- Example: If Thread A is waiting for disk I/O, Thread B can still run on another core.

4. Context Switching

- Requires **kernel intervention**, making it **slower** than user-level context switching.
- The kernel must save/restore **register states, stack pointers, and other thread-specific data**.

5. Function Calls & API Invocations

- When a KLT invokes a threads library function (e.g., `pthread_create()` in a kernel-supported implementation), it **triggers a system call**.
- Example: In Linux, `clone()` is the system call used to create threads.

6. Operating Systems Supporting KLTs Most modern general-purpose OSes support kernel-level threads, including:

- **Windows** (Win32 Threads)
- **Linux** (NPTL - Native POSIX Thread Library)

- **macOS** (based on Mach threads)
- **Solaris** (Unix-based thread model)

5.15.3.3 3.3 Advantages of Kernel-Level Threads

- **True Parallelism:** Can utilize **multiple CPU cores** effectively.
- **Better Responsiveness:** Blocking one thread does **not stall the entire process**.
- **Access to Kernel Services:** Can use **kernel-provided synchronization** (e.g., mutexes, semaphores).

5.15.3.4 3.4 Disadvantages of Kernel-Level Threads

- **Slower Context Switching:** Requires kernel mode transitions -> higher overhead.
- **Less Portable:** Thread management depends on the **OS kernel**, making programs less portable.
- **Higher Resource Usage:** Each KLT consumes **kernel resources** (e.g., TCB, stack space).

5.15.4 4. Comparison: User-Level Threads vs. Kernel-Level Threads

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Management	Managed by user-space library (no kernel support).	Managed by the kernel (requires system calls).
Kernel Awareness	Kernel unaware (treats process as single-threaded).	Kernel recognizes and schedules threads.
Blocking Behavior	Entire process blocks if one thread blocks.	Only the blocking thread stalls; others continue.
Context Switching	Fast (no kernel involvement).	Slow (requires kernel mode transition).
Multi-Core Support	No (kernel sees only one thread).	Yes (threads can run on different cores).
Portability	High (library-based, OS-independent).	Low (depends on OS kernel).
Synchronization	Limited to user-space mechanisms.	Can use kernel-provided primitives.
Examples	Pthreads (user-space), Java threads, Windows threads.	Linux NPTL, Windows Win32 threads, Solaris threads.

5.15.5 5. Summary of Key Concepts

1. User-Level Threads (ULT)

- Managed **entirely in user space** (no kernel support).
- **Faster context switching** but **no true parallelism**.
- **Blocking one thread blocks the whole process**.
- Examples: Pthreads (user-space), Java threads, Windows threads (legacy).

2. Kernel-Level Threads (KLT)

- Managed by the **kernel** (requires system calls).
- **Slower context switching** but **supports multi-core parallelism**.
- **Blocking one thread does not affect others**.
- Examples: Linux NPTL, Windows Win32 threads, macOS threads.

3. Hybrid Models (Not Covered in This Lecture)

- Some systems use a **combination** of ULTs and KLTs (e.g., **M:N model**), where multiple user threads map to fewer kernel threads.
- This approach balances **performance** and **parallelism** but introduces complexity.

5.15.6 6. Conclusion

- **User-level threads** are **lightweight and fast** but **lack true parallelism** and suffer from **blocking issues**.
- **Kernel-level threads** provide **better concurrency and multi-core support** but incur **higher overhead** due to kernel involvement.
- The choice between ULTs and KLTs depends on **application requirements** (e.g., need for parallelism vs. performance overhead).

5.16 What is Multicore programming

5.16.1 Introduction to Multicore Programming

- **Definition:** Multicore programming refers to the design and implementation of applications that leverage multiple processing cores (or processors) to execute tasks concurrently.
- **Core Idea:** Applications are divided into modular, independent tasks, each executed by a separate thread. When run on multicore/multiprocessor systems, these threads can execute **simultaneously and in parallel**, improving performance and resource utilization.

5.16.2 Key Concepts in Multicore Programming

5.16.2.1 1. Modular Application Design

- Applications are decomposed into **discrete tasks** (e.g., modules or components).
- Each task represents a logical unit of work (e.g., handling I/O, computations, user interface updates).
- **Example:** A web server may split tasks into:
 - Listening for client requests (Task 1).
 - Processing requests (Task 2).
 - Logging activities (Task 3).

5.16.2.2 2. Multithreaded Applications

- **Definition:** A program where multiple threads execute tasks concurrently within the same process.
- **Advantage:** Threads share the same memory space, reducing overhead compared to multiprocessing.
- **Execution Model:**
 - Each thread handles a specific task.
 - On a **multicore system**, threads can run on separate cores **in parallel**, enabling true concurrency.

5.16.2.3 3. Parallel Execution on Multicore Systems

- **Multicore/Multiprocessor Systems:** Modern CPUs contain multiple physical cores (e.g., dual-core, quad-core, or many-core processors).
- **Parallelism:** If the number of threads $\leq$ number of cores, threads can run **simultaneously** on different cores.
- **Outcome:**
 - **Computational Speedup:** Tasks complete faster due to parallel execution.
 - **Throughput Increase:** More work is done per unit time.

5.16.3 Benefits of Multicore Programming

5.16.3.1 1. Improved Resource Utilization

- **Single-Threaded Limitation:** Can only utilize **one core** at a time, leaving other cores idle.
- **Multithreaded Advantage:** Distributes workload across multiple cores, maximizing CPU usage.

- **Example:** A video rendering application can split frames across threads, rendering multiple frames in parallel.

5.16.3.2 2. Cost-Effective Scalability

- **Process vs. Thread Creation:**
 - **Process Creation:** High overhead (memory allocation, context switching, IPC mechanisms).
 - **Thread Creation:** Lower overhead (shares process memory, lighter context switches).
- **Scenario Comparison:**
 - **Single-Threaded Approach:** To utilize multiple cores, you must launch **multiple instances** of the program (each as a separate process), incurring high costs.
 - **Multithreaded Approach:** A single process with multiple threads can distribute work across cores **without process duplication**, reducing overhead.

5.16.3.3 3. Throughput at Lower Cost

- **Definition:** Throughput = Amount of work completed per unit time.
- **Multithreading Impact:**
 - More threads -> More cores utilized -> Higher throughput.
 - Lower resource cost (threads vs. processes) -> **Efficient scaling**.

5.16.4 Challenges and Considerations

5.16.4.1 1. Task Independence

- **Requirement:** Tasks must be **independent** to achieve true parallelism.
 - **Independent Tasks:** No dependencies; can execute in any order.
 - **Dependent Tasks:** Must execute sequentially (e.g., Task B requires output from Task A).
- **Consequence of Dependencies:**
 - If tasks are dependent, parallelism is **not possible**-threads will execute serially, negating multicore benefits.
- **Example:**
 - **Independent:** Calculating different rows of a matrix in parallel.
 - **Dependent:** A pipeline where Task 2 processes output from Task 1.

5.16.4.2 2. Programming Complexity

- **Synchronization:** Threads may need to coordinate (e.g., via locks, semaphores) to avoid race conditions or data corruption.
- **Debugging:** Concurrency bugs (e.g., deadlocks, starvation) are harder to diagnose than in single-threaded programs.

5.16.5 Multicore Programming vs. Single-Threaded Programming

Aspect	Single-Threaded	Multithreaded (Multicore)
Core Utilization	1 core	Multiple cores (if threads are independent)
Performance	Limited by single-core speed	Potential speedup via parallelism
Resource Overhead	Low (single process)	Moderate (thread management)
Scalability	Poor (cannot leverage additional cores)	High (scales with core count)
Cost of Concurrency	High (requires multiple processes)	Low (threads within one process)

5.16.6 Practical Implications

1. When to Use Multicore Programming:

- Applications with **independent, parallelizable tasks** (e.g., scientific computing, servers, media processing).
- Scenarios requiring **high throughput** (e.g., web servers handling multiple requests).

2. When to Avoid:

- Tasks with **strong dependencies** (e.g., sequential algorithms).
- Applications where **synchronization overhead** outweighs parallelism benefits.

5.16.7 Summary of Key Points

- Multicore programming enables **parallel execution** of tasks via multithreading.
- **Independent tasks** are essential for leveraging multicore systems.
- Multithreading improves **resource utilization** and **throughput** at a lower cost than multiprocessing.
- **Trade-offs:** Increased complexity in synchronization and debugging.
- **Goal:** Write applications that **maximize core usage** while minimizing overhead.

5.17 What is thread

5.17.1 Introduction to Threads

- A **thread** is defined as a **lightweight process**.
- Threads enable **concurrent execution** within a single process.
- Threads are the **basic unit of CPU utilization** in systems supporting multi-threading.

5.17.2 Single-Threaded vs. Multi-Threaded Processes

5.17.2.1 Single-Threaded Process

- Contains **only one thread of execution**.
- Can perform **only one task at a time**.
- Example: A process with a single thread executing sequentially.

5.17.2.2 Multi-Threaded Process

- Contains **multiple threads of execution** (or threads of control).
- Can perform **multiple tasks simultaneously** (concurrently or in parallel, depending on CPU availability).
- **Parallel execution** is possible if multiple processors are available.
- **Key distinction:** Threads within a process share resources but maintain independent execution paths.

5.17.3 Components of a Thread

Each thread consists of **four unique components**:

1. Thread ID (TID)

- A **unique identifier** assigned to each thread within a process.
- Used by the system to distinguish between threads.

2. Program Counter (PC)

- Holds the **address of the next instruction** to be executed by the thread.

3. Register Set

- Contains **temporary values** used during computation.
- Each thread has its own set of registers to maintain execution state.

4. Stack

- Used for **function calls, local variables, and return addresses**.
- Each thread has its **own private stack** to avoid interference.

5.17.3.1 Shared Components Among Threads in a Process While the above components are unique to each thread, threads within the **same process** share: - **Code section** (executable instructions). - **Data section** (global and static variables). - **Files and resources** allocated to the process (e.g., open files, I/O devices).

5.17.4 Visual Representation: Single-Threaded vs. Multi-Threaded Process

5.17.4.1 Single-Threaded Process Structure

- **Code section:** Contains executable instructions.
- **Data section:** Contains global and static variables.
- **Files/Resources:** Allocated by the OS (e.g., open files).
- **Stack:** Single stack for the sole thread.
- **Registers:** Single set of registers for the sole thread.

5.17.4.2 Multi-Threaded Process Structure

- **Shared components** (same as single-threaded):
 - Code section.
 - Data section.
 - Files/Resources.
- **Per-thread components:**
 - **Three threads**, each with:
 - * A **private stack**.
 - * A **private set of registers**.
 - * A **program counter**.
 - * A **thread ID (TID)**.

Key Observation: - Threads **share** the process's code, data, and resources. - Threads **do not share** stacks, registers, or program counters (these are thread-specific).

5.17.5 Thread Representation in the System

5.17.5.1 Thread Control Block (TCB)

- Analogous to the **Process Control Block (PCB)** for processes.
- A **kernel-level data structure** that stores thread-specific information.
- Used by the OS to **manage and schedule threads**.

5.17.5.1.1 Components of a Thread Control Block (TCB)

1. Thread Identifier (TID)

- Unique ID assigned to the thread (also called **tid**).
- Distinguishes the thread within the process and system.

2. Stack Pointer

- Points to the **thread's private stack** within the process's address space.

3. Program Counter (PC)

- Stores the **address of the next instruction** to be executed by the thread.

4. Thread State

- Indicates the current state of the thread (e.g., **running**, **ready**, **waiting**).
- Similar to process states but specific to the thread's execution context.

5. Register Values

- Stores **temporary values** used by the thread during computation.
- Includes general-purpose registers, flags, and other CPU-specific registers.

6. Pointer to the Process Control Block (PCB)

- Links the thread to the **process that created it**.
- Helps the OS identify the parent process and its resources.

7. Pointers to Child Threads (if applicable)

- If the thread creates additional threads, the TCB contains **pointers to their TCBs**.
- Enables hierarchical thread management.

5.17.6 Summary of Key Concepts

- A **thread** is a lightweight process enabling concurrent execution within a process.
- **Single-threaded processes** perform one task at a time; **multi-threaded processes** perform multiple tasks concurrently/parallelly.
- Each thread has **unique components**: Thread ID, Program Counter, Register Set, and Stack.
- Threads within a process **share** code, data, and resources.
- Threads are represented in the system via a **Thread Control Block (TCB)**, which stores thread-specific information for management by the OS.

5.18 Why is thread lightweight

5.18.1 Introduction

- Threads are often referred to as “**lightweight processes**” in operating systems.
- This lecture explores the reasons behind this classification by comparing threads and processes in terms of:
 - Resource sharing.
 - Creation overhead.
 - Context switching efficiency.
 - Communication mechanisms.

5.18.2 Thread vs. Process: Resource Sharing

5.18.2.1 Shared Resources Among Threads Threads belonging to the **same process** share the following resources: 1. **Code Section** - The executable instructions of the program. 2. **Data Section** - Global and static variables. 3. **OS-Level Resources** - Open files, network connections, etc. 4. **Address Space** - All threads of a process operate within the **same memory address space**. - Memory locations allocated to the process are **shared** among all its threads.

5.18.2.2 Examples of Shared Resources

- **Global Variables**
 - If a process declares a global variable `x`, all threads of that process **access the same instance** of `x`.
 - Threads **do not** have private copies of global variables.
- **Dynamically Allocated Variables (Heap Memory)**
 - Memory allocated using `malloc()`, `new`, etc., is **shared** among all threads of the process.

5.18.2.3 Non-Shared Resources (Thread-Specific) Despite sharing many resources, threads maintain **private** copies of: 1. **Stack** - Each thread has its **own stack** for function calls and local variables. - Local variables in

functions are **not shared** among threads. 2. **Registers** - Each thread has its own set of CPU registers. 3. **Program Counter (PC)** - Tracks the execution point of the thread independently.

5.18.3 Why is Process Creation “Heavyweight”?

Process creation is **more expensive** than thread creation due to the following reasons:

5.18.3.1 1. Memory Allocation Overhead

- A **new process** requires:
 - Allocation of a **new address space**.
 - Setup of **memory segments** (code, data, stack, heap).
 - Involvement of the **Memory Management Unit (MMU)** to manage virtual-to-physical memory mappings.
- This involves **significant OS intervention** and **hardware support**.

5.18.3.2 2. Independent Resource Duplication

- Each process has its **own copy** of:
 - Open files.
 - Network sockets.
 - IPC (Inter-Process Communication) mechanisms (if used).
- **No inherent resource sharing** between processes unless explicitly set up (e.g., shared memory, pipes).

5.18.4 Why is Thread Creation “Lightweight”?

Thread creation is **less expensive** than process creation because:

5.18.4.1 1. No New Address Space Allocation

- Threads **share the existing address space** of their parent process.
- Example:
 - If a process already has **3 threads** and a **4th thread** is created:
 - * The new thread **reuses the same memory space** as the existing threads.
 - * **No need** to allocate a new address space or involve the MMU.

5.18.4.2 2. Minimal OS Overhead

- The OS only needs to:
 - Allocate a **new stack** for the thread.
 - Initialize **thread-specific registers** (e.g., Program Counter, Stack Pointer).
 - Update the **Process Control Block (PCB)** to include the new thread’s state.
- **No duplication** of process-level resources (e.g., open files, memory segments).

5.18.4.3 3. Faster Execution Startup

- Since threads **share the process’s memory**, they can **immediately access** global data and code.
- **No loading delay** (unlike processes, which may require loading executable files from disk).

5.18.5 Inter-Thread Communication (ITC) vs. Inter-Process Communication (IPC)

Aspect	Threads (Same Process)	Processes (Different Processes)
Communication Mechanism	Direct memory access (shared variables)	Explicit IPC (pipes, message queues, shared memory)
Synchronization Needed	Yes (e.g., mutexes, semaphores)	Yes (but more complex due to separate address spaces)
Overhead	Low (no OS intervention for data sharing)	High (OS must manage communication channels)
Example	Thread A modifies <code>global_x</code> ; Thread B reads it.	Process A writes to a pipe; Process B reads from it.

5.18.5.1 Advantages of Thread Communication

- **No need for IPC mechanisms** (e.g., pipes, message passing).
- **Faster data sharing** (since memory is shared by default).
- **Simpler synchronization** (though still required for thread safety).

5.18.5.2 Disadvantages of Thread Communication

- **Race conditions** can occur if threads access shared data without synchronization.
- **Debugging complexity** increases due to non-deterministic thread execution.

5.18.6 Context Switching: Process vs. Thread

5.18.6.1 Process Context Switching

1. Steps Involved:

- Save the **entire state** of the current process (PCB: registers, memory maps, etc.).
- Load the **entire state** of the new process.
- **Switch to a different address space** (requires MMU intervention).
- Flush and reload **TLB (Translation Lookaside Buffer)** entries.

2. Overhead:

- **High** due to:
 - Address space switching.
 - Cache and TLB invalidation.
 - OS-level bookkeeping (e.g., updating process tables).

5.18.6.2 Thread Context Switching (Same Process)

1. Steps Involved:

- Save **thread-specific state** (registers, stack pointer, PC).
- Load the **new thread's state**.
- **No address space change** (threads share the same memory).

2. Overhead:

- **Low** because:
 - No MMU involvement.
 - No TLB flushes.
 - Only **stack and registers** need to be switched.

5.18.6.3 Comparison Table

Factor	Process Context Switch	Thread Context Switch
Address Space Change	Yes (full MMU intervention)	No (same address space)
State Saved/Loaded	Entire PCB (memory maps, file descriptors, etc.)	Only registers, stack pointer, PC
TLB Impact	Flushed (high overhead)	No change (low overhead)
Time Complexity	High (microseconds range)	Low (nanoseconds range)
OS Involvement	Extensive (scheduler, MMU, etc.)	Minimal (only thread control block updates)

5.18.6.4 Key Takeaway

- **Thread context switching is ~10-100x faster** than process context switching.
- **Less time wasted** in switching, leading to **better performance** in multithreaded applications.

5.18.7 Summary of Why Threads Are Lightweight

1. **Resource Sharing**
 - Threads share the **same address space**, reducing memory overhead.
 - No duplication of process-level resources (e.g., open files).
2. **Creation Efficiency**
 - No need to allocate a **new address space** (unlike processes).
 - Only **stack and registers** are initialized per thread.
3. **Communication Efficiency**
 - **Direct memory access** (no IPC mechanisms required).
 - Faster data exchange between threads.
4. **Context Switching Efficiency**
 - **No address space change** -> faster switches.
 - **Lower OS overhead** (only thread-specific state is saved/loaded).

5.18.8 Conclusion

- Threads are called “**lightweight**” because:
 - They **share resources** with their parent process.
 - They **require less OS intervention** for creation and context switching.
 - They **enable efficient communication** without explicit IPC.
- **Trade-off:** While threads are lightweight, they introduce **synchronization challenges** (e.g., race conditions, deadlocks).
- **Use Case:** Ideal for **parallel tasks** within the same process (e.g., web servers, GUI applications, multi-threaded computations).